

基于特征工程、K-means 聚类、生存分析和随机森林的 NIPT 的 时点选择与胎儿的异常判定

摘要

NIPT (Non-invasive Prenatal Test, 即无创产前检测) 是一种早期检测胎儿健康状况的产前技术, 其准确性十分依赖与胎儿染色体浓度, 本文旨在通过建立数学模型, 分析相关的关键指标与染色体浓度关系, 并且根据此模型来确定不同孕妇群体所对应的最佳的检测时机和对于异常胎体判定的方法, 用以运用到实际中来减少检测与治疗的潜在风险。

对于问题一, 该问题分别构建起机器学习中的随机森林、GBM、特征选择后的随机森林模型。对正常的数据集中的胎儿 Y 染色体浓度与孕周(GA)和 BMI 的关系进行分析。最终随机森林预测结果近似成一个“可写公式”的形式, 类似拟合一条曲线公式。公式为 $Y = 0.289606 - 0.007702 \times GA - 0.005927 \times BMI - 0.000190 \times Age + 0.000275 \times GA \times BMI + 0.000026 \times GA^2$, 而特征选择后的随机森林模型拟合度为 $R^2 = 0.8525$, 具有优良显著性。

对于问题二, 本题中采用 K-means 聚类, 确定 K=3 后把孕妇按照 BMI 区间分为三组, 通过组内胎儿 Y 染色体浓度达标时间的 80 百分位数来确定每一组孕妇所对应的最佳的 NIPT 时点。最终聚类 0 组的 BMI 区间为 [20.7, 31.53], 最佳检测时间为 20.29 周; 聚类 1 组的 BMI 区间为 [35.42, 46.88], 最佳检测时间为 23.68 周; 聚类 3 组的 BMI 区间为 [31.55, 35.38], 最佳检测时间为 20.29 周。而在误差分别为 5%、10%、15% 时, 各组 BMI 最小波动范围分别为 15.94 到 30.40、19.32 到 35.23、31.00 到 37.32; 最大波动范围为 29.97 到 35.78、30.38 到 47.11、35.09 到 58.83; 最佳检测周波动范围为 22.72 到 23.78、21.55 到 24.40、23.68 到 24.95。

对于问题三, 该问题使用聚类的方法的, 在综合考虑身高、体重、年龄、BMI 等多因素同时, 再引入生存分析模型 (Kaplan-Meier 曲线) [1], 同样以 80 百分位数来确定最佳 NIPT 时点, 最终得到聚类 0, 1 组的 BMI 分布分别为 33.33 ± 3.29 、 31.40 ± 2.33 ; 年龄分布分别为 28.73 ± 3.95 、 29.12 ± 3.39 ; 身高分布分别为 165.43 ± 3.19 、 157.36 ± 3.47 ; 体重分布分别为 91.16 ± 8.74 、 77.72 ± 5.85 。而聚类 0, 1 组最佳 NIPT 时点分别为 23.38 周、20.56 周。对关键指标采取 5% 扰动, 对结果会产生 $\pm 8\%$ 的相对误差。

对于问题四, 采取随机森林的机器学习方法, 以题干中提及的 21、18、13 号染色体非整倍体判定结果为标签, 再综合 X 染色体 Z 值、GC 含量、读段数、BMI 等因素作为标签特征来进行模型的训练。最终, 根据异常概率公式计算得到异常概率值。异常概率公式为 $(\sum(w_i \times feature_i) + b)$, 而对于异常概率超过 50% 的个体, 我们将其视为异常。

最后, 各项模型都具有一定的准确性、科学性与稳定性, 同时在论文中提出的根据 BMI 区间分组的策略与最佳 NIPT 时点的选择对于实际实践应用也有一定的指导作用。

关键字: NIPT 特征工程 K-means 聚类 生存分析 机器学习分类

一、问题重述

1.1 问题背景

NIPT (Non-invasive Prenatal Test, 即无创产前检测) 是现在常见的一种产前检测技术, 其目的为通过早期的检测来确定胎儿的健康状况。根据临床经验, 畸型胎儿主要有唐氏综合征、爱德华氏综合征和帕陶氏综合征, 这三种体征分别由胎儿 21 号、18 号和 13 号“染色体游离 DNA 片段的比例”(简称“染色体浓度”)是否异常决定。NIPT 在检测中的准确性主要是根据胎儿体染色体(男胎为 XY, 女胎为 XX)的浓度来进行判断。一般情况下, 在 10 周 25 周间便可以进行对胎儿染色体浓度的检测, 若男胎的 Y 染色体的浓度大于等于 4%、女胎的 X 染色体浓度并无异常, 那么就可以认为 NIPT 的结果基本准确。在现实情况下, 对于不健康的胎儿应该尽早发现, 否则便会有治疗窗口期缩短的风险。实际中发现, 早期(12 周内)发现的风险较低; 中期(13 周 27 周)发现的风险较高; 晚期(28 周以后)发现的风险极高。除此之外, 男胎 Y 染色体浓度与孕妇的孕周以及身体质量指数(BMI)紧密相关, 但由于每个孕妇的年龄与 BMI 以及孕情存在着个体差异。以防一刀切处理, 提高 NIPT 的准确性以最大发挥其效用, 因此通常情况会根据孕妇的 BMI 值来分组以便分别确定合适的 NIPT 的时点, 同时会有多次采血或者一次采血多次检测的情况来增加检测结果的可靠性。

1.2 问题要求

问题 1 试分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标的相关特性, 给出相应的关系模型, 并检验其显著性。

问题 2 临床证明, 男胎孕妇的 BMI 是影响胎儿 Y 染色体浓度的最早达标时间(即浓度达到或超过 4% 的最早时间)的主要因素。试对男胎孕妇的 BMI 进行合理分组, 给出每组的 BMI 区间和最佳 NIPT 时点, 使得孕妇可能的潜在风险最小, 并分析检测误差对结果的影响。

问题 3 男胎 Y 染色体浓度达标时间受多种因素(身高、体重、年龄等)的影响, 试综合考虑这些因素、检测误差和胎儿的 Y 染色体浓度达标比例(即浓度达到或超过 4% 的比例), 根据男胎孕妇的 BMI, 给出合理分组以及每组的最佳 NIPT 时点, 使得孕妇潜在风险最小, 并分析检测误差对结果的影响。

问题 4 由于孕妇和女胎都不携带 Y 染色体, 重要的是如何判定女胎是否异常。试以女胎孕妇的 21 号、18 号和 13 号染色体非整倍体(AB 列)为判定结果, 综合考虑 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素, 给出女胎异常的判定方法。

二、问题分析

2.1 问题一分析

对于问题一，首先要进行数据筛选：NIPT 的准确性主要由胎儿性染色体（男胎 XY，女胎 XX）浓度判断。通常孕妇的孕期在 10 周到 25 周之间可以检测胎儿性染色体浓度，且如果男胎的 Y 染色体浓度达到或高于 4%、女胎的 X 染色体浓度没有异常，则可认为 NIPT 的结果是基本准确的，否则难以保证结果准确性要求。随后需要考虑孕周、BMI、Y 染色体浓度这三个指标，通过绘制热力图来观察其关系，使用机器学习中的随机森林、GBM、特征选择后的随机森林模型，分别求取 R2 值最后对比得出结果。

2.2 问题二分析

对于问题二，考虑使用聚类方法基于 BMI 对孕妇进行分组，利用肘部法则来确定 K 值，从而获得了三个分组，对于每组找出 Y 染色体浓度大于等于 4% 的百分位点作为最佳孕周。同时，还要考虑到治疗窗口期缩短的风险来进行最佳检测时间的选择。最终得出模型，最后进行聚类误差验证。

2.3 问题三分析

对于问题三，考虑使用 BMI、身高、体重、年龄进行 min-max 标准化，以这四个数据为特征进行聚类，从而得到分组。获得各项特征均值以及一定误差范围，最后再使用生存模型对数据集进行分析，同样考虑治疗窗口期缩短的风险来选择得到最佳检测时间 Kaplan-Meier 曲线，最后用检验误差对结果的影响。

2.4 问题四分析

对于问题四，考虑 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素，首先使用随机森林的机器学习方法训练模型，以女胎数据集为训练集和验证集训练模型并进行交叉验证。再使用 python 中的 sklearn 库训练得到模型和 AUC 等数据，得到女胎异常标准，以此作为判定方法的判断准则。

三、模型假设

为简化问题，本文做出以下假设：

- 假设 1：题中所有畸形胎儿类型都为唐氏综合征、爱德华氏综合征和帕陶氏综合征其中之一。

- 假设 2: 检测失败的数据不会影响变量之间的关联性, 可以看作为一种随机事件。
- 假设 3: 假设孕妇的胎体潜在的风险仅仅与发现异常的孕周有关, 与其它题目中未提及因素无关。
- 假设 4: 假设在构建 Y 染色体浓度预测模型时, BMI 与孕妇的孕周相互独立、互不影响。

四、符号说明

符号	说明	单位/范围
f_{log_i}	对数变换特征函数	—
x_i	原始特征	(由不同特征自身决定) —
Y	Y 染色体浓度	%
α_0	GAM 模型的整体截距	%
$f(GA)$	Y 染色体浓度随孕周变化的非线性趋势的平滑函数	—
$g(BMI)$	Y 染色体浓度随 BMI 变化的非线性趋势的平滑函数	—
ε	随机误差项	%
β_i	模型需要从数据中拟合得到的回归系数	$i \in [0, 5]$
BMI	孕妇身体质量指数	kg/m^2
GA	孕周数	周
R^2	决定系数	—
y_i	真实预测值	—
$\bar{y}$	观测均值	—
n	样本数	个
SSE	所有数据点与其所属簇的聚类中心 (质心) 之间距离的平方和	原始数据单位的平方
x_{1i}	第 i 个数据点	—
c_j	第 j 个簇的质心	—
Age	年龄	岁
S_t	时间 t 时的生存概率	%
d_i	在时间 i 发生事件 (达标) 的个体数	个

n_i	在时间 i 处于风险中的个体数（尚未达标且未被截尾）	个
$\hat{y}$	预测值	—
$\arg \max_c$	使函数取得最大值的参数 c	—
$\frac{1}{T}$	归一化系数（1/分类器数量）	—
$\sum_{t=1}^T$	从 t=1 到 T 的求和	—
$\mathbb{I}$	I，表示指示函数（Indicator function）	—
$h_t(x) = c$	第 t 个分类器预测为类别 c	—
Z	标准化的统计量	—
X	待检测样本中目标染色体的相对计数比例	%
μ	正常对照群体中该染色体计数比例的均值	%
σ	正常群体中该比例的标准差	%
DP	异常概率	%
σ_1	sigmoid 函数	—
w_i	各特征的权重	%
$feature_i$	标准化后的特征值	—
b	偏置项	—

五、问题一的模型的建立和求解

5.1 模型建立

采用机器学习的方法对多个指标的相关性进行分析，利用随机森林、GBM、特征选择后的随机森林三种模型，并比较优化模型之间的性能差距。其中，最终选择的优化模型的目标函数为：

$$Y = \beta_0 + \beta_1 \times GA + \beta_2 \times BMI + \beta_3 \times Age + \beta_4 \times GA \times BMI + \beta_5 \times GA^2 \quad (1)$$

此公式即为关系模型，之后再求取 R^2 值得到拟合度来反应显著性。

5.2 模型求解

Step1: 对于问题一，我们初步设想决定使用线性回归的方法，我们取 BMI、Y 染色体浓度、和孕周为关键指标。首先对这三个因素进行相关性的研究：

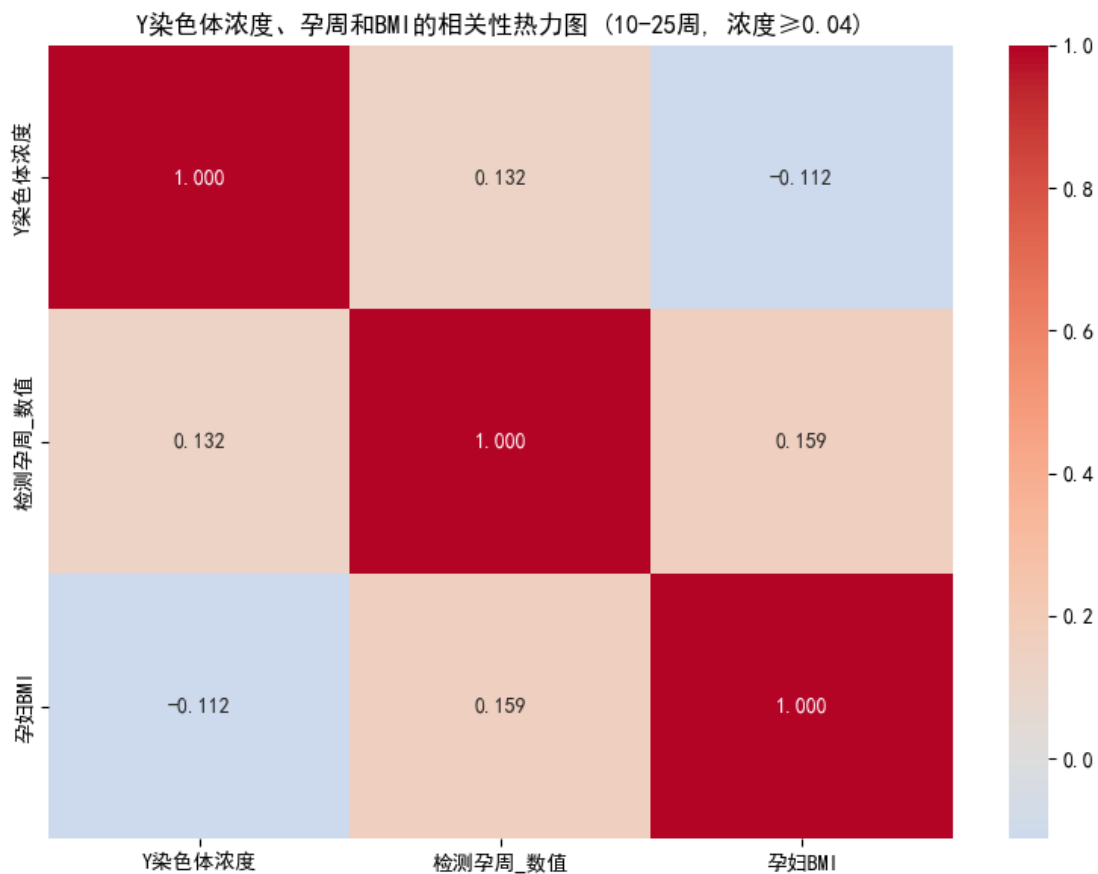


图 1 特征相关性图

图中颜色越深代表两个特征的相关性越强。

Step2: 随后将 Y 染色体浓度作为因变量，将孕周以及 BMI 作为自变量。然后，对所有男胎数据进行最小二乘法估计得到一次直线：

$$Y = \beta_0 + \beta_1 \times BMI + \beta_2 \times GA \quad (2)$$

代码计算得出使得残差平方和最小的各项系数 (β_i , 其中 $i \in [0, 2]$), 分别为 $\beta_0 = 0.1182$ 、 $\beta_1 = -0.0018$ 、 $\beta_2 = 0.0011$ 。根据此由代码生成图像：

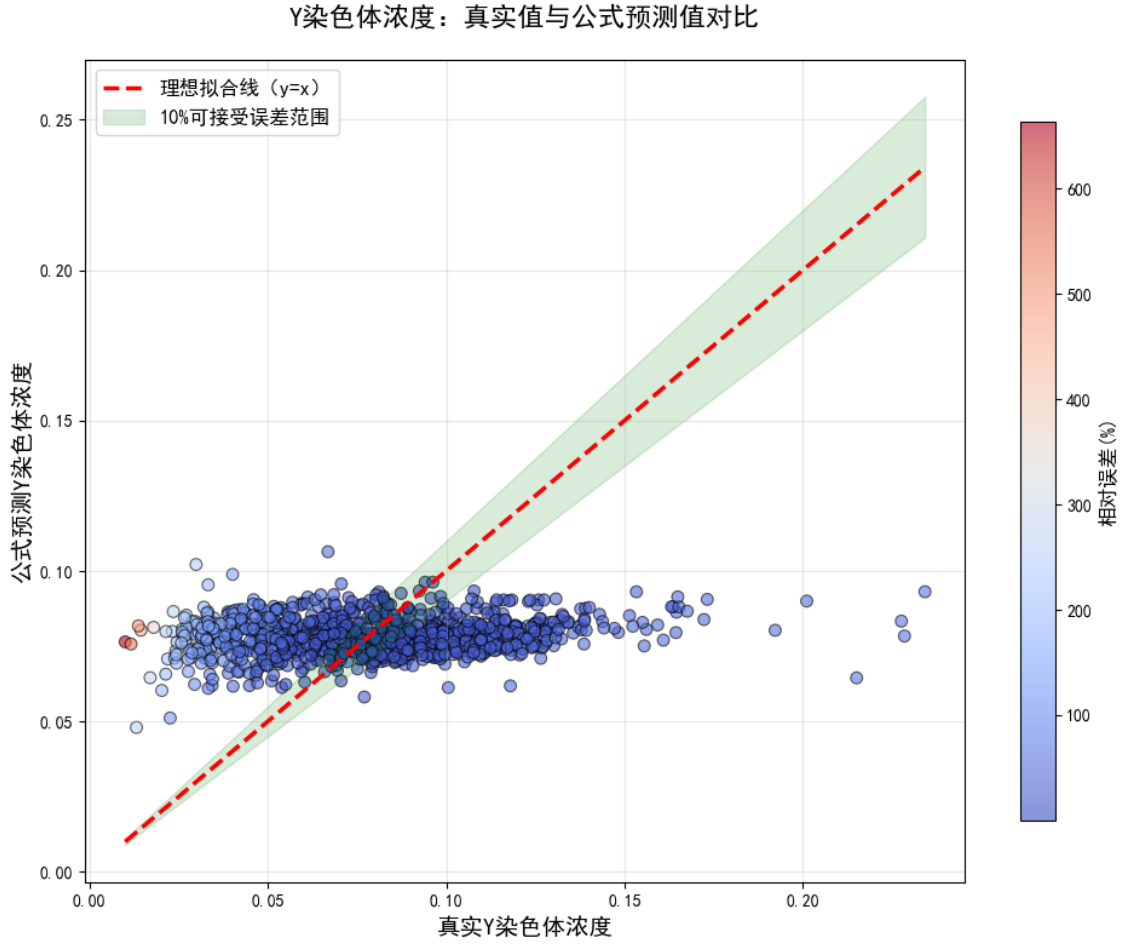


图2 一次直线拟合效果图

可看出一次直线拟合效果较差，因此再对其采用添加多项式处理，添加多项式如下：

$$Y = \beta_0 + \beta_1 \times BMI + \beta_2 \times GA + \beta_3 \times BMI^2 + \beta_4 \times BMI \times GA + \beta_5 \times GA^2 \quad (3)$$

代码计算得出使得残差平方和最小的各项系数（ β_i ，其中 $i \in [0, 5]$ ），分别为 $\beta_0 = 0.035630$ 、 $\beta_1 = 0.007317$ 、 $\beta_2 = -0.006954$ 、 $\beta_3 = -0.000177$ 、 $\beta_4 = 0.000153$ 、 $\beta_5 = 0.000086$ 。使用代码绘图如下：

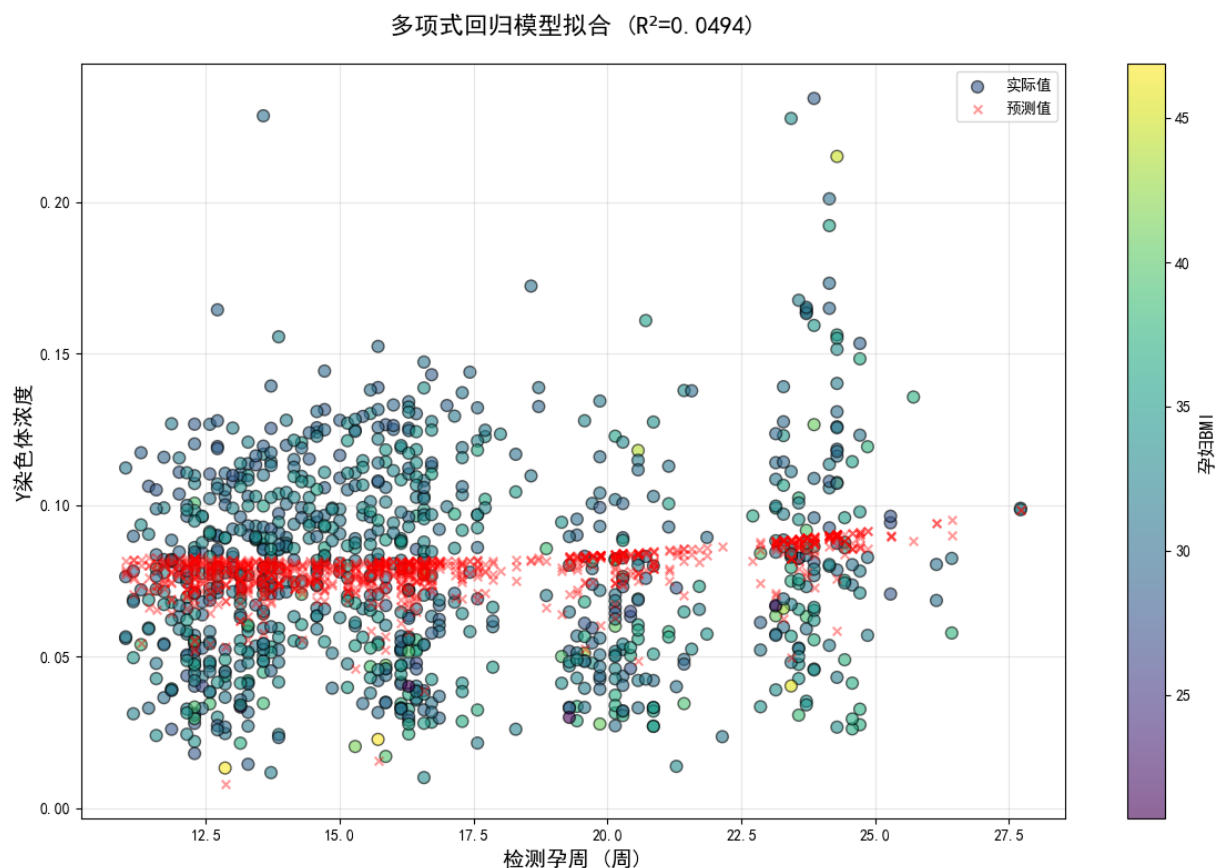


图3 多项式回归拟合图

对于多项式回归，我们使用代码求取 R^2 的值发现与 1 相差较大，表明多项式回归拟合效果较差。另外，从医学的角度看，机器学习模型提供了可解释性以及严谨性，同时可以处理非线性关系，因此我们最终采用机器学习的方法求取 Y 染色体浓度和孕周以及 BMI 的相关特性。

Step3: 最后我们采用三种不同的方法优化机器学习模型，即随机森林、GBM 和特征选择后的随机森林，其中，最终选择特征选择后的随机森林模型的优化函数为 (1)。随之而后将男胎数据集按照 7: 3 的比例分为数据集和验证集以确保模型的准确性。利用 python 中的 sklearn 库训练模型，使用其中的随机森林计算函数、GBM 工具和多特征提取工具对模型进行优化，最终得到相关特征热力图以及相关特征重要性排名图：

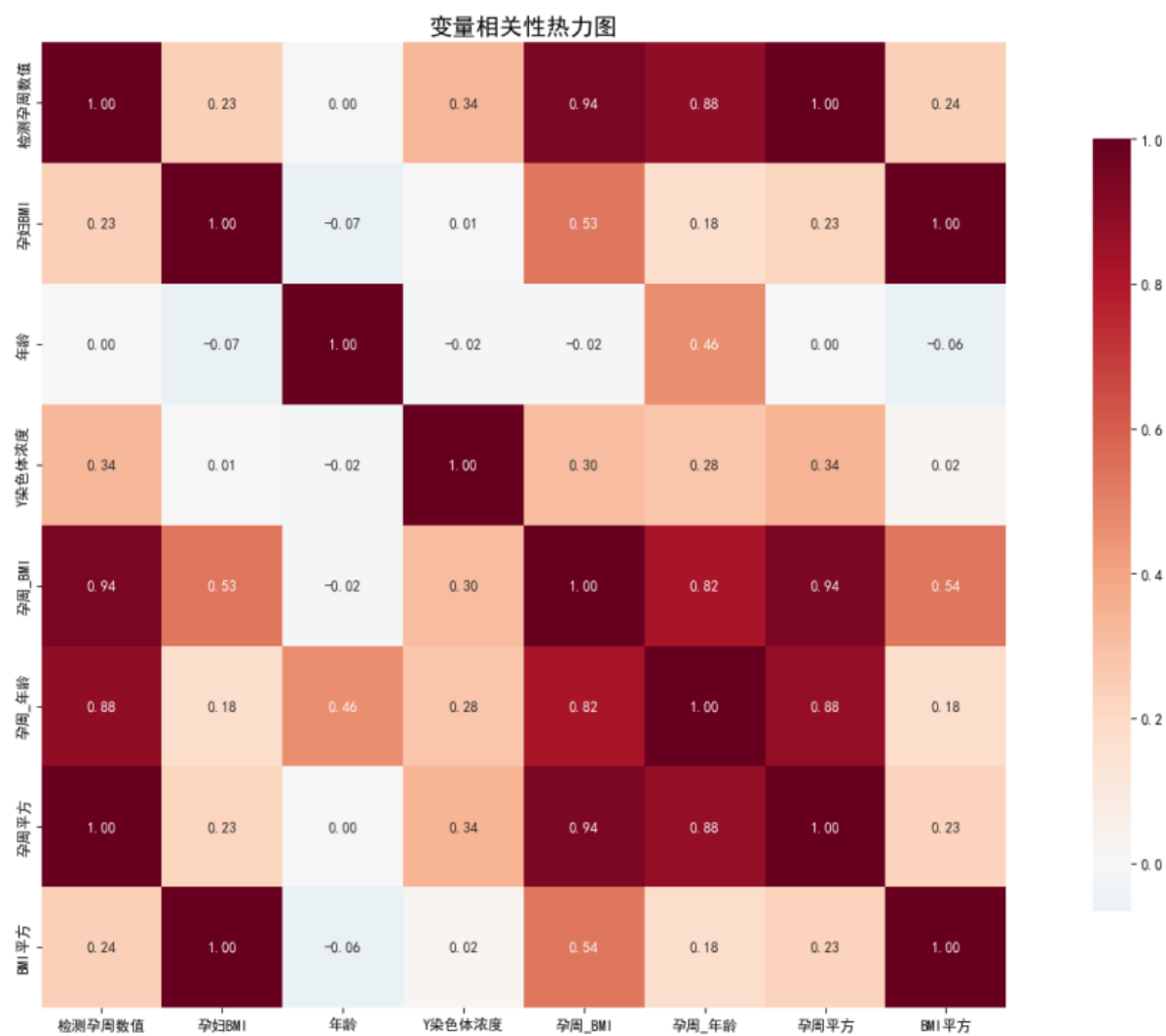


图4 相关特征热力图

以及相关特征重要性排名图：

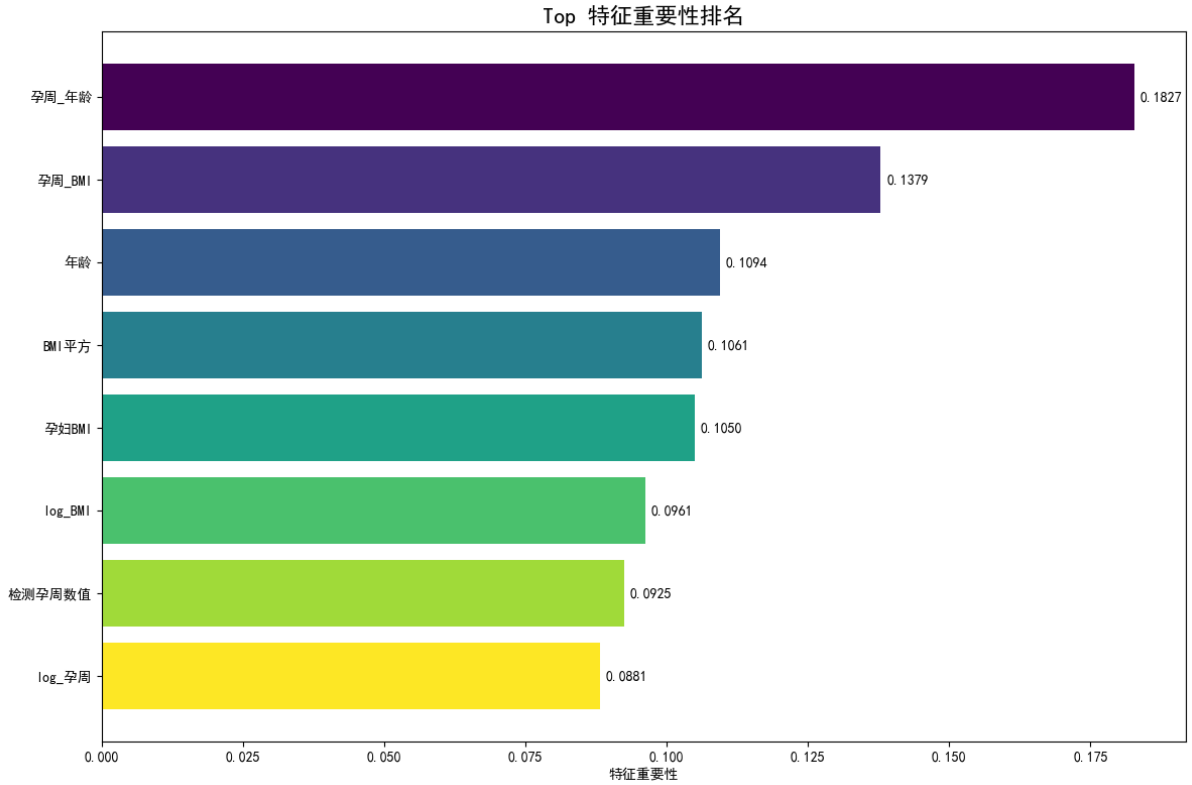


图 5 相关特征重要性排名图

在热力图中，颜色越深代表该行该列的特征相关性越大。在重要性图中，该特征数值越大代表 Y 染色体浓度与该特征相关性越大。

5.3 求解结果

特征选择后的随机森林模型是非线性模型，本质上是树模型，不能直接生成一个数学公式表达式，则用 python 代码将特征选择后的随机森林模型近似成一个“可写公式”的形式，类似一条曲线公式 (1)，求解各项系数后得到关系模型为 $Y = 0.289606 - 0.007702 \times GA - 0.005927 \times BMI - 0.000190 \times Age + 0.000275 \times GA \times BMI + 0.000026 \times GA^2$ 。

为检验该模型的显著性，我们分别代入数据对随机森林、GBM 和特征选择后的随机森林通过决定系数公式：

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4)$$

求取 R^2 值，得到的结果如下：

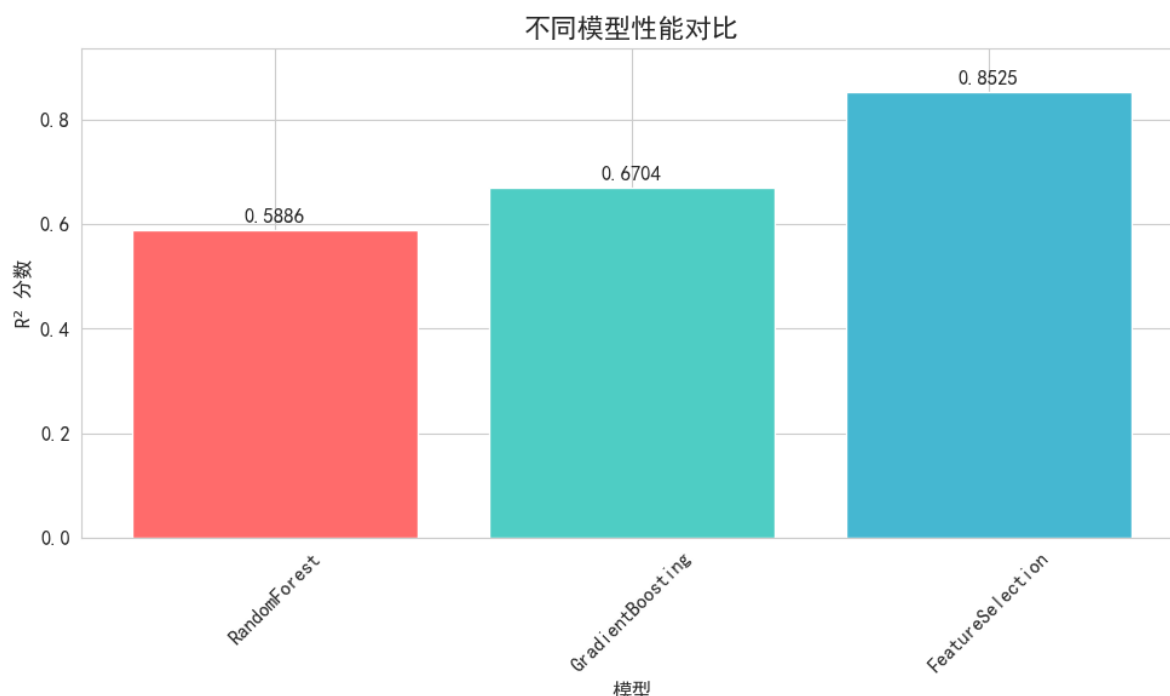


图 6 不同模型的 R^2 值图

上图中的 R^2 值越接近 1 代表拟合效果越好，我们所使用的特征选择后的随机森林值到达了 0.8525，远高于使用随机森林模型或是 GBM 模型。对于线性回归模型，特征选择后的随机森林模型性能更是提升了 68%。

六、问题二的模型的建立和求解

6.1 模型建立

对于问题二，我们决定使用 K-means 聚类的方法对所有数据进行分组，利用目标函数：

$$SSE = \sum_{i=1}^n \sum_{j=1}^k d(x_{1i}, c_j)^2 \quad (5)$$

该公式为误差平方和公式， $d(x_{1i}, c_j)^2$ 为一个孕妇的 BMI 值与聚类中心的 BMI 值之差的平方。带入对应数据，让 K 遍历 2 到 10，直到获得使 SSE 最小的 K 值。

6.2 模型求解

Step1: 首先引用公式 (5) 遍历 K 值得到 K=3 时为模型复杂度（3 个组）和聚类效果（SSE 显著降低）之间的最佳平衡点。以 BMI 为关键指标由代码生成分组如下：

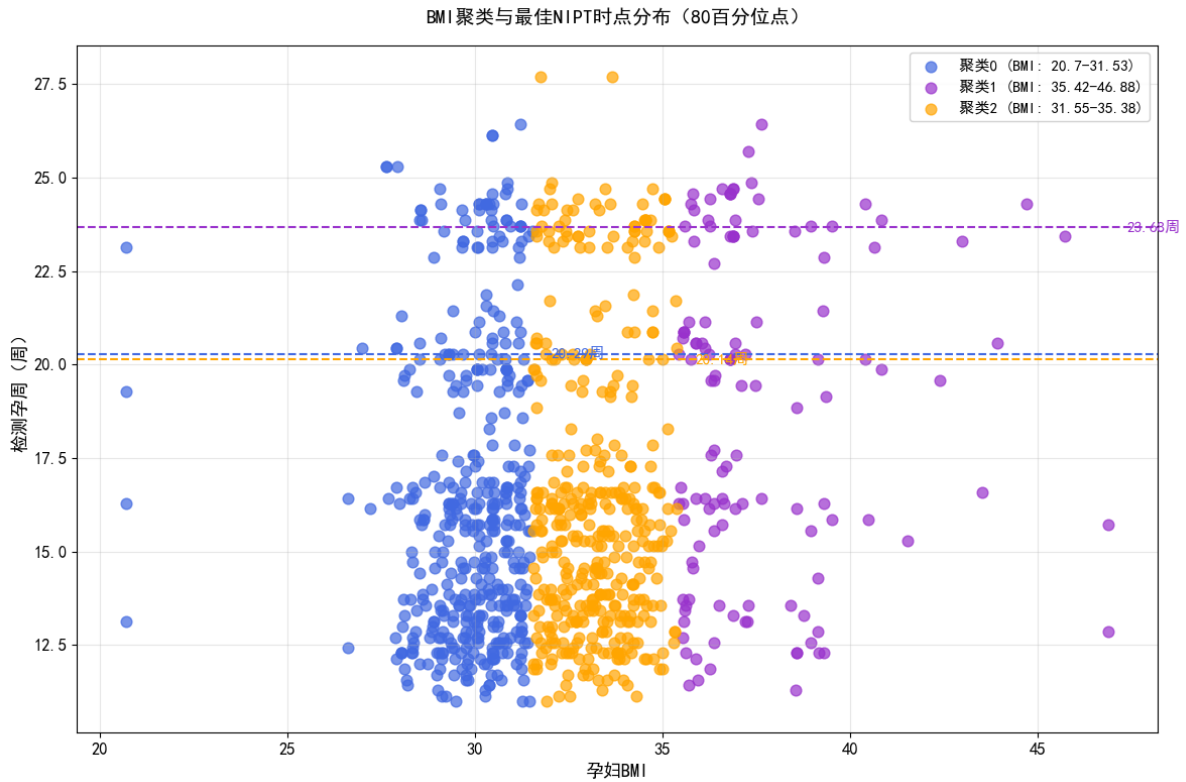


图 7 男胎聚类散点图

上图中不同颜色的点代表不同的聚类，图中的虚线即为八十百分位点，对应的纵坐标即为最佳 NIPT 检测时间。

Step2: 对于每个聚类的数据集取百分位点作为最佳 NIPT 检测时间。考虑到取用九十百分位点可能导致时间过晚，错过最佳 NIPT 检测时间，因此取用八十百分位点作为最佳 NIPT 检测时间点，确保既不会因为过晚检测错过检测时间点，也不会因为达标人数过低导致检测准确度下降。

6.3 求解结果

最终得到各个聚类的最佳 NIPT 检测时间：

表 1 各聚类组最佳 NIPT 检测时间

聚类组别	BMI 区间	最佳检测时间 (80 百分位)
聚类 0	[20.7, 31.53]	20.29 周
聚类 1	[35.42, 46.88]	23.68 周
聚类 2	[31.55, 35.38]	20.29 周

对于误差验证，我们分别取用了 5%、10% 和 15% 的误差形成聚类，观察误差对结

果的扰动：

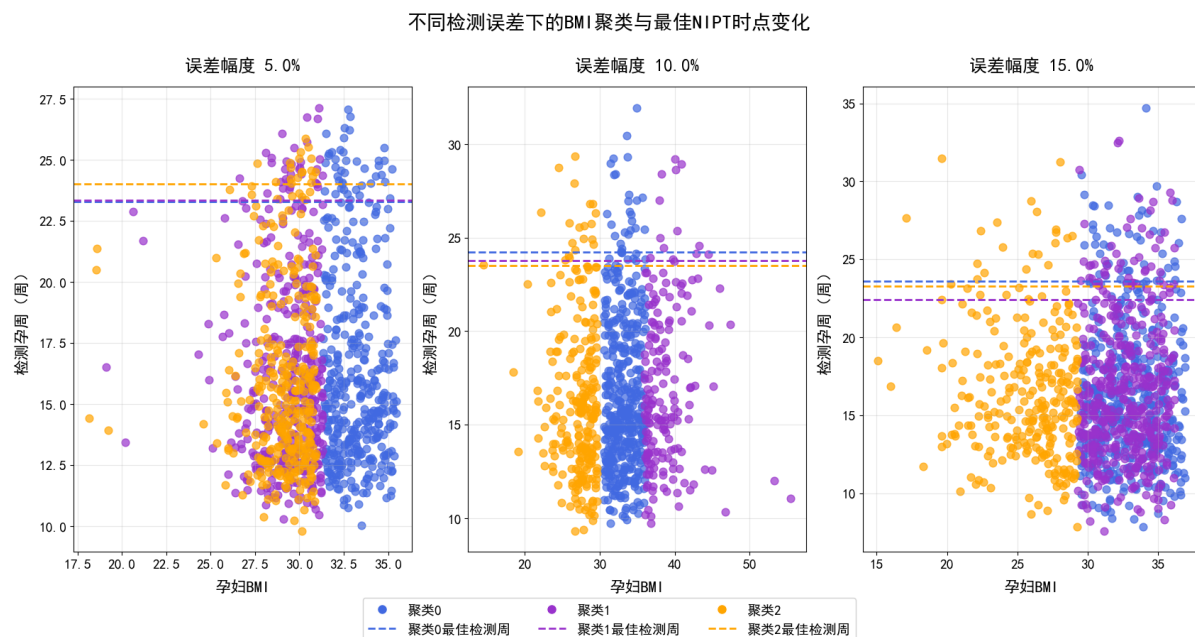


图 8 不同检测误差下的 BMI 聚类与最佳 NIPT 时点变化图

如上图所示，误差会对聚类结果以及 NIPT 最佳检测周数产生不小的扰动，在误差分别为 5%、10%、15% 时，各组 BMI 最小波动范围分别为 15.94 到 30.40、19.32 到 35.23、31.00 到 37.32；最大波动范围为 29.97 到 35.78、30.38 到 47.11、35.09 到 58.83；最佳检测周波动范围为 22.72 到 23.78、21.55 到 24.40、23.68 到 24.95。

七、问题三的模型的建立和求解

7.1 模型建立

对于问题三，我们采用生存模型的方法，以孕周作为时间，Y 染色体浓度作为事件，通过以目标函数：

$$S(t) = \prod_{i=1}^t \left[1 - \frac{d_i}{n_i} \right] \quad (6)$$

为根据， d_i 为时间 t 时的生存概率， d_i 为在时间 i 发生事件（达标）的个体数。带入对应数据，用代码生成 Kaplan-Meier 曲线。在曲线上取八十百分位点得到最佳 NIPT 检测时间。

7.2 模型求解

Step1: 考虑到 Y 染色体浓度及其符合时间-事件对应的特性，因此我们考虑使用生存模型进行求解。首先，以身高、体重、BMI 以及年龄为关键指标对男胎数据集进行聚类。得到聚类结果如下：

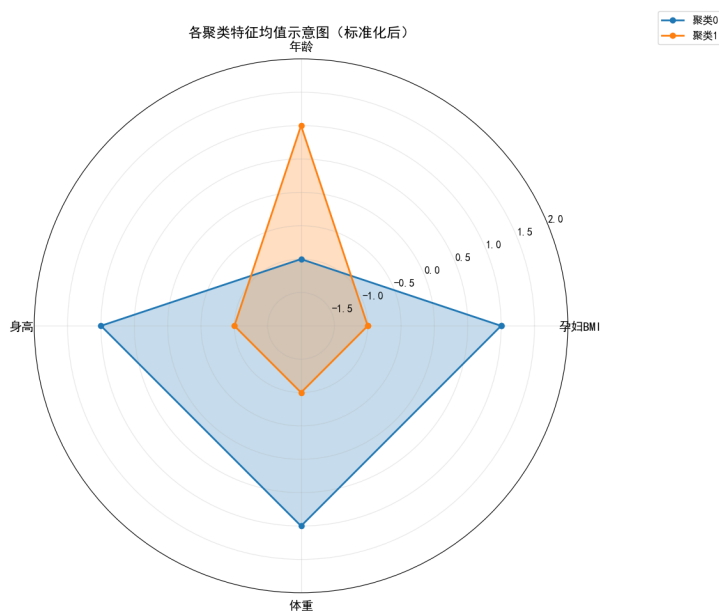


图9 各个聚类的特征均值雷达图

雷达图中的数据代表不同聚类的特征均值。

Step2: 使用 python 中的 lifelines 库以孕周为时间，Y 染色体浓度为事件，引入公式 (6) 生成生存模型并求得 Kaplan-Meier 曲线，如下：

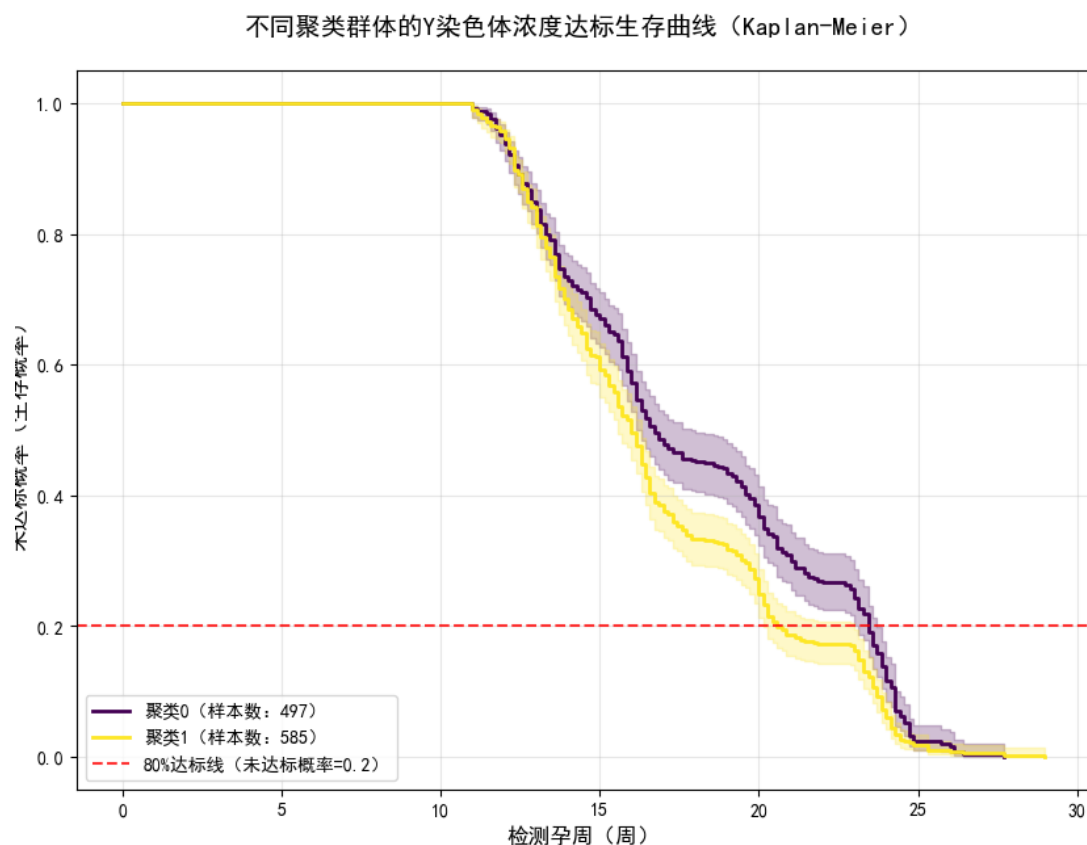


图 10 不同聚类 Y 染色体浓度达标生存曲线

图中纵坐标代表不达标率，即生存模型中的生存率，横坐标代表检测孕周，即生存模型中的时间。图中红色虚线代表 80% 达标线，即八十百分位数。由于选取九十百分位数有可能导致时间太晚导致部分孕妇错过低风险检测时间，因此我们选取 80 百分位数作为最佳检测时间。

7.3 求解结果

得到结果：

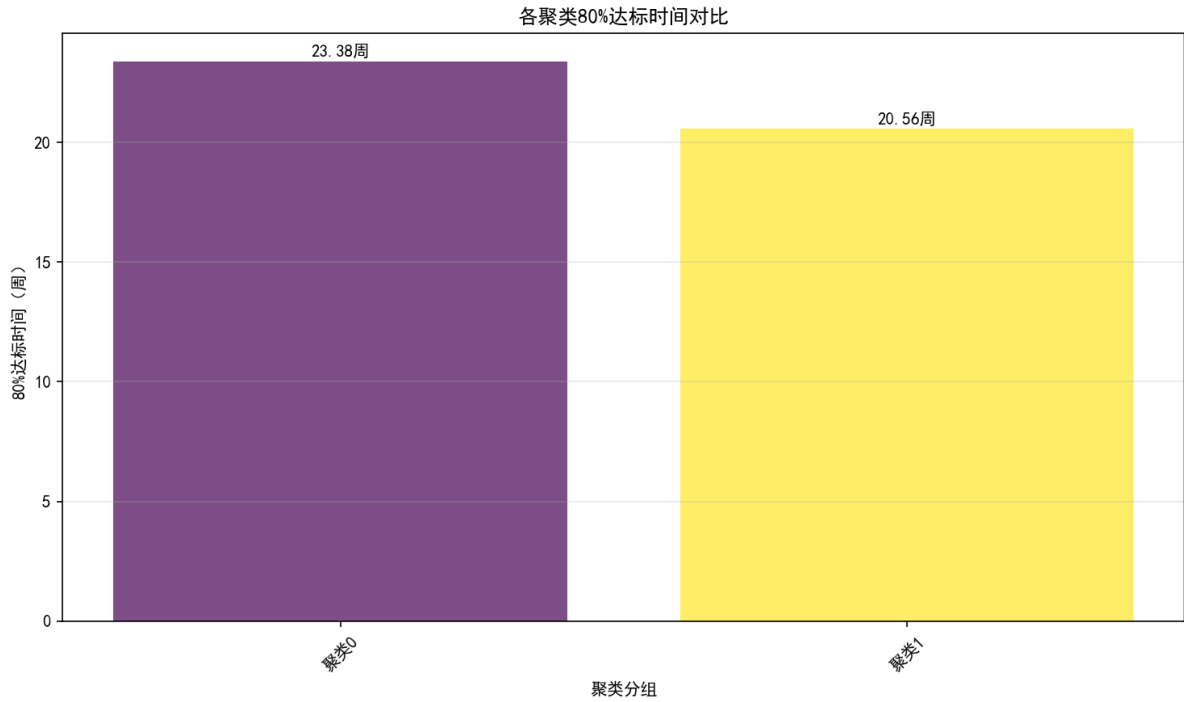


图 11 各个聚类别 80% 达标时间

对于误差检测，我们采用特征稳定性检查。最终得到聚类 0，1 组的 BMI 分布分别为 33.33 ± 3.29 、 31.40 ± 2.33 ；年龄分布分别为 28.73 ± 3.95 、 29.12 ± 3.39 ；身高分布分别为 165.43 ± 3.19 、 157.36 ± 3.47 ；体重分布分别为 91.16 ± 8.74 、 77.72 ± 5.85 。而聚类 0，1 组最佳 NIPT 时点分别为 23.38 周、20.56 周。对关键指标采取 5% 扰动，对结果会产生 $\pm 8\%$ 的相对误差。

八、问题四的模型的建立和求解

8.1 模型建立

对于问题四，我们采用机器学习训练模型的方式对女胎异常进行预测。先用二分法处理染色体的非整倍体问题，选择特征训练，利用两个目标函数：

$$\hat{y} = \arg \max_c \frac{1}{T} \sum_{t=1}^T \mathbb{I}(h_t(x) = c) \quad (7)$$

该公式为随机森林中的多数投票法（分类问题），随后再进行机器训练与学习来求解。

$$DP = \sigma(\sum(w_i \times feature_i) + b) \quad (8)$$

该公式为异常概率公式，用以判断女胎是否异常。

8.2 模型求解

Step1: 首先, 我们通过 Z 值 (Z-score) 公式:

$$Z = \frac{X - \mu}{\sigma} \quad (9)$$

求得 Z 值, 然后我们再将所有的变量 (年龄, 体重, 孕妇 BMI, 身高, 原始读段数, 在参考基因组上比对的比例, 重复读段的比例, 唯一比对的读段数, GC 含量, 被过滤掉读段数的比例, 13 号染色体的 Z 值, 18 号染色体的 Z 值, 21 号染色体的 Z 值, X 染色体的 Z 值, X 染色体浓度, 13 号染色体的 GC 含量, 18 号染色体的 GC 含量, 21 号染色体的 GC 含量, 怀孕次数, 生产次数) 作为特征引用公式 (7) 使用 python 中的 sklearn 库对模型进行训练, 得出不同变量的重要性程度, 并从中选取十个最重要的制成图表:

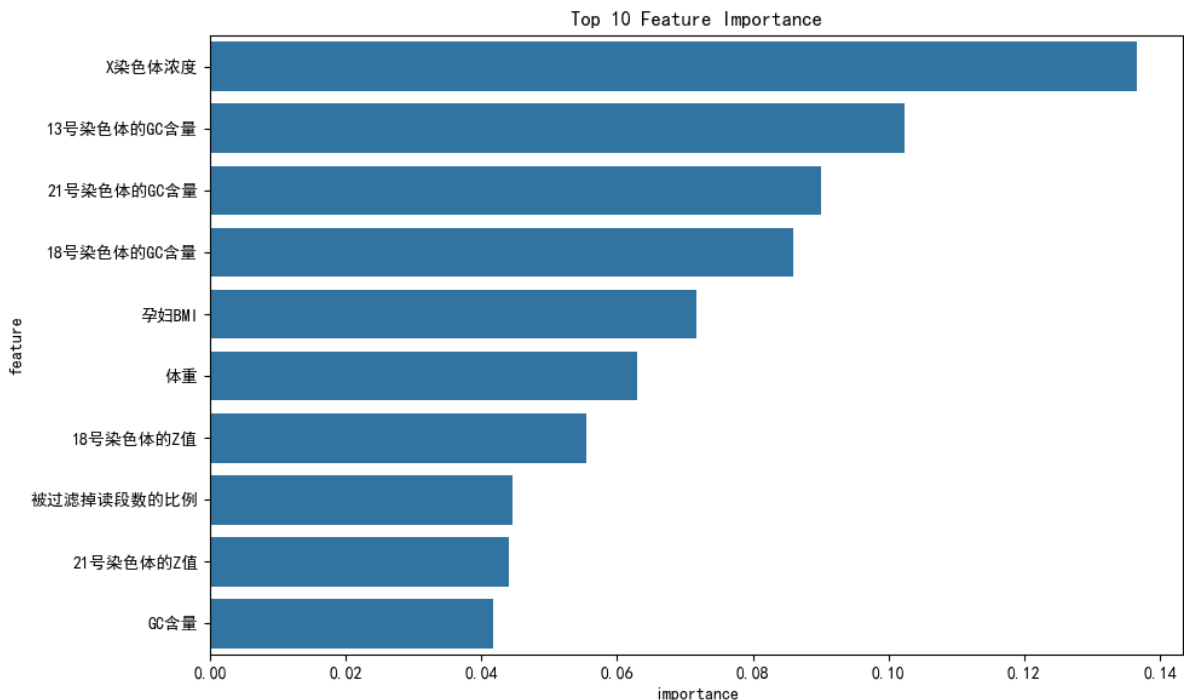


图 12 十个对判断异常最重要的指标图

由此可看出, 前十个最重要的指标分别为: X 染色体浓度、13 号染色体的 GC 含量、21 号染色体的 GC 含量、18 号染色体的 GC 含量、孕妇 BMI、体重、18 号染色体的值、被过滤掉段数的比例、21 号染色体的 Z 值、GC 含量。

Step2: 对于每个独立的数据, 模型根据训练结果得出其发生异常的概率:

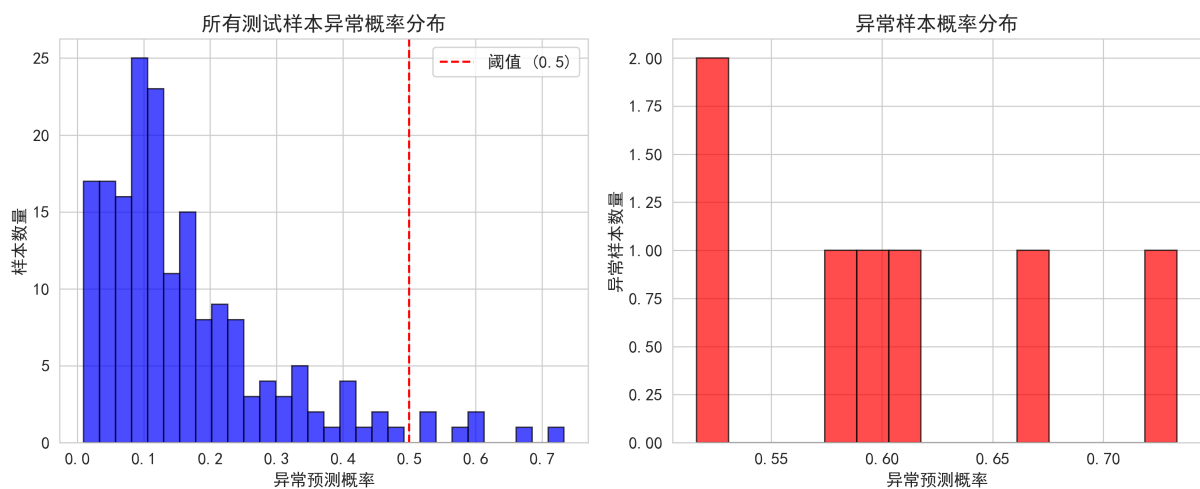


图 13 女胎数据发生异常的概率分布图

上图展示了，女胎数据发生异常的概率分布。

8.3 求解结果

找出训练集中预测为异常的样本，将它们的特征去标准化，恢复到原始数值。取最重要的四个特征绘制正常样本与异常样本关键指标的对比图如下所示：

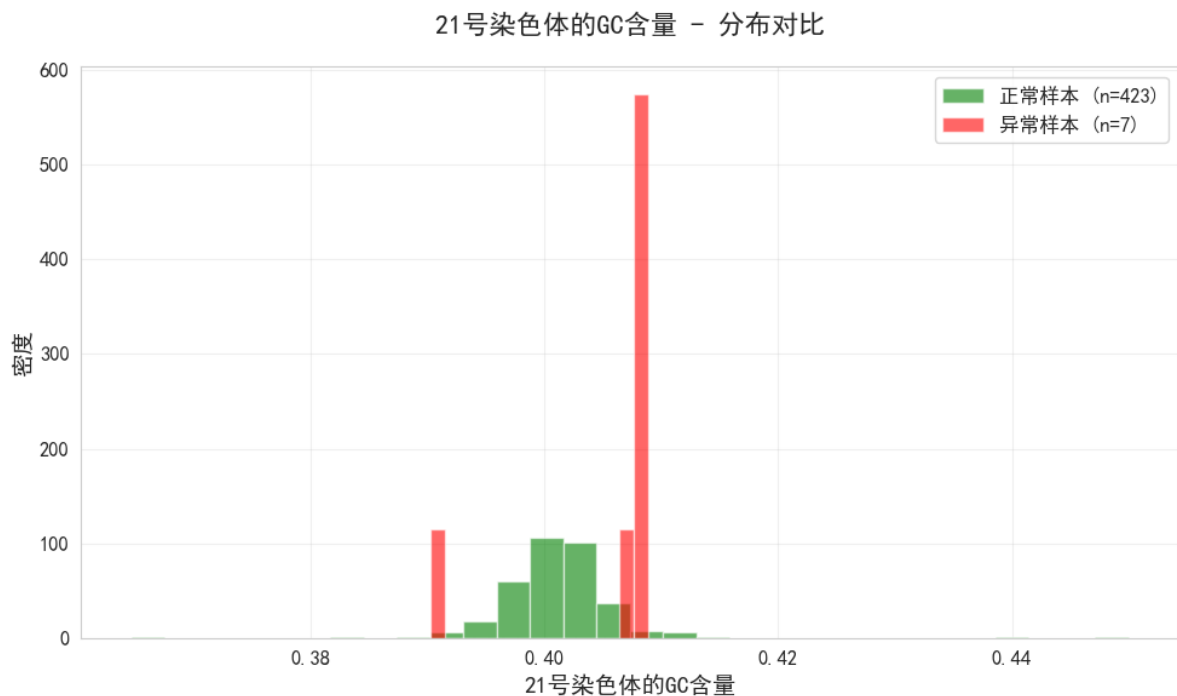


图 14 21 号染色体的 GC 含量图

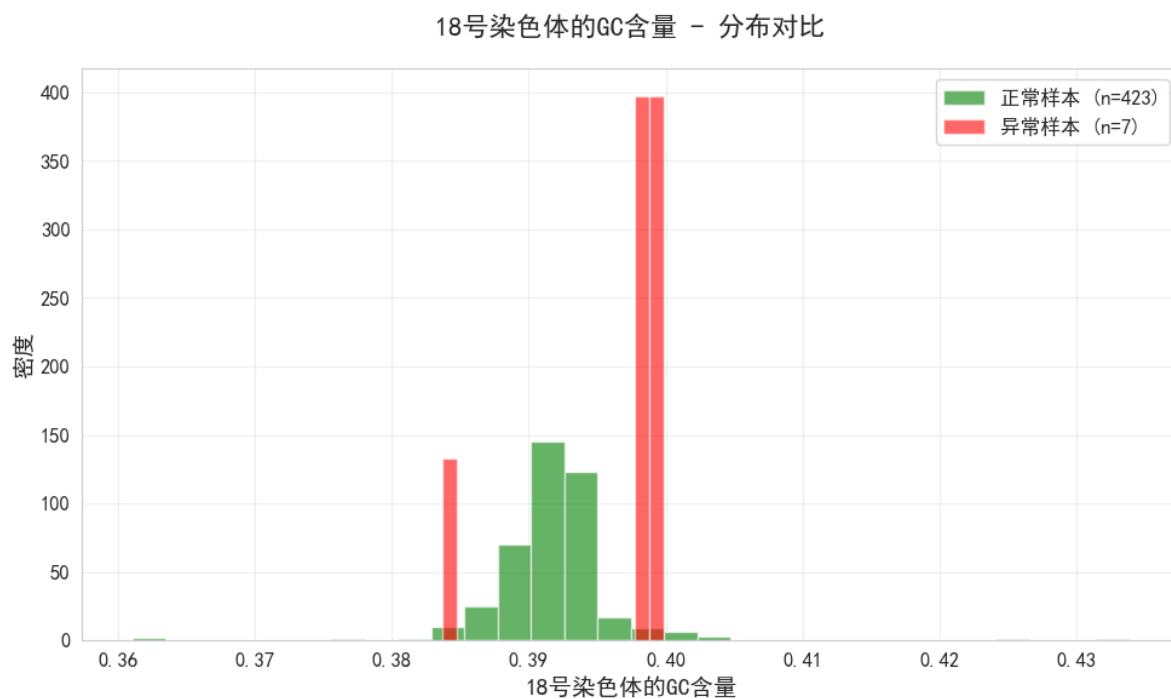


图 15 18 号染色体的 GC 含量图

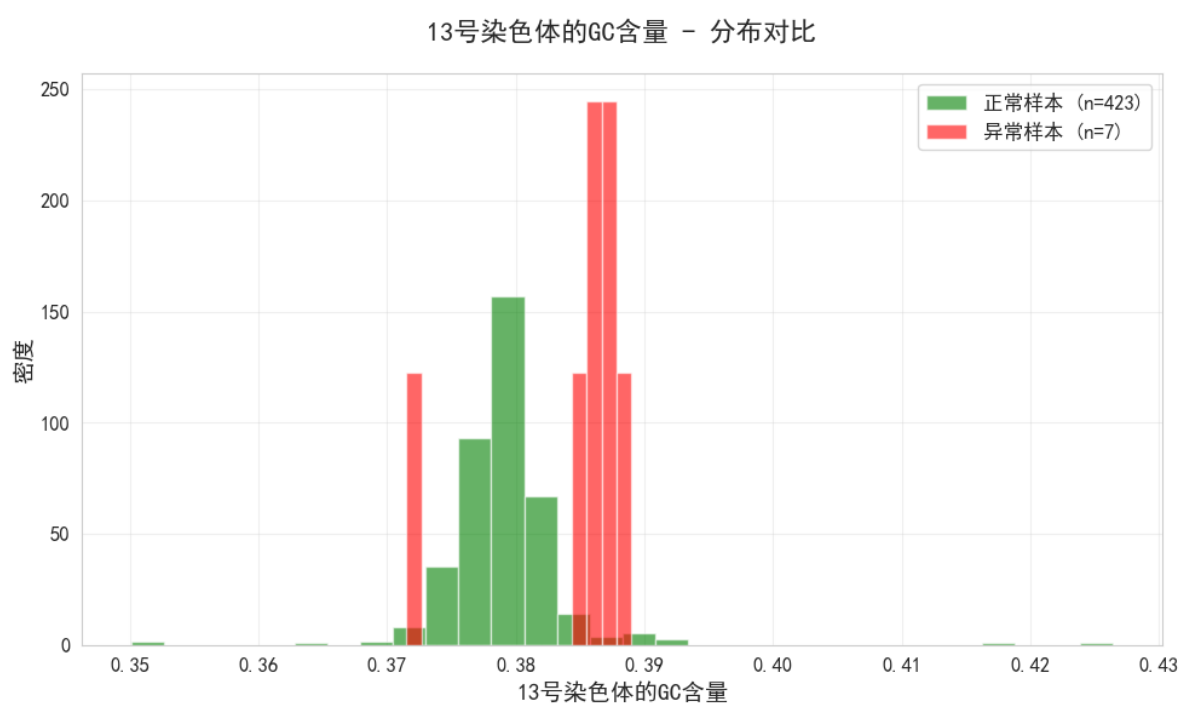


图 16 13 号染色体的 GC 含量图

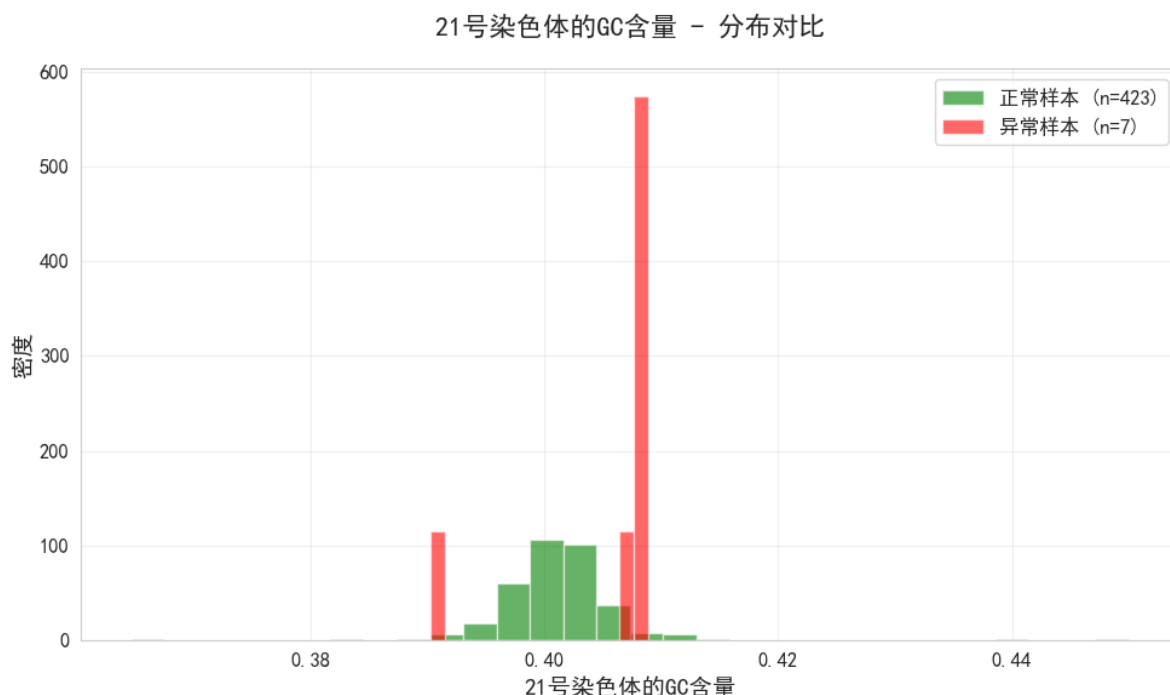


图 17 21 号染色体的 GC 含量图

以上四张图分别显示了 X 染色体浓度、13 号、18 号和 21 号染色体 GC 含量在正常样本和异常样本之间的不同，这表明 NIPT 检测异常的关键指标为 X 染色体浓度、13 号、18 号和 21 号染色体 GC 含量。又根据图 13 与异常概率公式 (8)，对于异常概率超过 50% 的个体，我们将其视为异常女胎。

九、模型的分析与检验

9.1 灵敏度分析

在问题一与三的处理上，我们对 BMI 与 GA 引入了 5% 以及 10% 的随机波动。结果表明，问题一与问题三的模型输出的变化幅度要小于扰动幅度（此处相对误差大约为 8%），所以可以认为问题一与问题三的模型对于数据小范围的波动并不敏感，整体具有一定的稳定性。

在问题二的处理上，我们在聚类分析过程中，改变了聚类初始中心点还增添了噪声点。在结果上发现各 BMI 分组所确定的最佳 NIPT 时间节点只有些许的偏移，因此同样可以说聚类的分组是具有一定稳定性和稳健性的。

在问题四的处理上，我们对分类模型中靠后的特征逐步移除来加用以训练，从此发现模型的性能在移除后五个特征后开始有了明显下滑，这说明模型对特征集合的构成具有一定的容错性。

9.2 误差分析

本次在开始便对数据进行了预处理，对于明显异常的数据记录进行了剔除，因此整体上减少了数据噪声对模型训练的影响，但在后续模型的拟合上仍然会具有一定的误差，这些误差有源于孕妇个体差异没能完全考虑而造成的，也有模型本身与数据不能完美契合带来的小误差。不过最终的各个模型，在整体上依旧是稳定可靠的。

十、模型的评价

10.1 模型的优点

- 优点 1：本次模型具有一定的综合性，使用了多种模型来处理对应的不同问题，例如在问题三中把聚类与生存分析结合起来。不但使得分组清晰，还使得时间点的选择清晰科学。
- 优点 2：本次模型还具有一定的科学性，问题三种的生存分析较为贴切问题中达标时间规定和治疗窗口缩短的风险的场景。
- 优点 3：本次模型还具有实用与稳定性，本次模型对于孕妇 BMI 区间分组有着清晰的划分，同时这样的模型在面对情况变化以及数据波动时有一定的稳定性。

10.2 模型的缺点

- 缺点 1：该模型无法对孕妇个体更多因素的差异做出反应与调整，具有一定的局限性。
- 缺点 2：该模型对于数据有着一定的依赖性，因此模型的泛化能力无从考证。

参考文献

- [1] Rich, J. T., Neely, J. G., Paniello, R. C., Voelker, C. C., Nussenbaum, B., & Wang, E. W. (2010). A practical guide to understanding Kaplan-Meier curves. *Otolaryngology—Head and Neck Surgery*, **143**(3), 331–336.
- [2] DeepSeek, DeepSeek-R1-0528, 深度求索（DeepSeek）, 2025-09-05–2025-09-07.
- [3] 豆包, 无具体版本与型号, 字节跳动, 2025-09-05–2025-09-07.

附录 A 文件列表

文件名	功能描述
q1.py	问题一程序代码
q2.py	问题二程序代码
q3.py	问题三程序代码
q4.py	问题四程序代码

附录 B 代码

q1.py

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.ensemble import RandomForestRegressor,
    GradientBoostingRegressor
6 from sklearn.model_selection import GridSearchCV
7 from sklearn.metrics import r2_score
8 from sklearn.feature_selection import SelectFromModel
9 import re
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # ===== 中文乱码解决方案 =====
14 def setup_chinese_font():
15     """设置中文字体，解决乱码问题"""
16     import matplotlib as mpl
17     from matplotlib import font_manager
18     import platform
19     import os
20
21     # 重置matplotlib默认设置
22     mpl.rcParams.update(mpl.rcParamsDefault)
23
```

```

24 # 设置样式
25 sns.set_style("whitegrid")
26 plt.rcParams['figure.figsize'] = [10, 6]
27 plt.rcParams['font.size'] = 12
28 plt.rcParams['axes.unicode_minus'] = False
29
30 # 根据不同操作系统设置字体
31 system = platform.system()
32 font_paths = []
33
34 if system == "Windows":
35     font_paths = [
36         "C:/Windows/Fonts/simhei.ttf",
37         "C:/Windows/Fonts/msyh.ttc",
38         "C:/Windows/Fonts/simsun.ttc",
39     ]
40 elif system == "Darwin":
41     font_paths = [
42         "/System/Library/Fonts/PingFang.ttc",
43         "/System/Library/Fonts/Hiragino Sans GB.ttc",
44     ]
45 else:
46     font_paths = [
47         "/usr/share/fonts/truetype/wqy/wqy-microhei.ttc",
48     ]
49
50 # 尝试加载字体文件
51 for font_path in font_paths:
52     if os.path.exists(font_path):
53         try:
54             font_prop = font_manager.FontProperties(fname=
font_path)
55             font_manager.fontManager.addfont(font_path)
56             plt.rcParams['font.family'] = [font_prop.
get_name(), 'DejaVu Sans', 'Arial']

```

```

57         print(f"□ 成功设置字体: {font_prop.get_name()}
    ")
58         return True
59     except:
60         continue
61
62     # 如果字体文件加载失败, 尝试使用字体名称
63     font_names = [
64         'SimHei', 'Microsoft YaHei', 'SimSun',
65         'PingFang SC', 'Hiragino Sans GB', 'WenQuanYi Micro
    Hei',
66         'DejaVu Sans', 'Arial'
67     ]
68
69     for font_name in font_names:
70         try:
71             plt.rcParams['font.family'] = [font_name, 'sans-
    serif']
72             fig, ax = plt.subplots()
73             ax.text(0.5, 0.5, '测试中文', fontsize=12, ha='
    center')
74             plt.close(fig)
75             print(f"□ 成功设置字体: {font_name}")
76             return True
77         except:
78             continue
79
80     print("□ 无法设置中文字体, 将使用英文显示")
81     plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial', '
    sans-serif']
82     return False
83
84 # 调用字体设置函数
85 setup_chinese_font()
86

```



```

87 # ===== 数据处理函数 =====
88 def convert_gestational_week(week_str):
89     """将'11w+6'格式的孕周转换为数值"""
90     if pd.isna(week_str):
91         return np.nan
92
93     if isinstance(week_str, str):
94         match = re.match(r'(\d+)w(?:\+(\d+))?', week_str)
95         if match:
96             weeks = int(match.group(1))
97             days = int(match.group(2)) if match.group(2) else
0
98             return weeks + days / 7.0
99         else:
100             try:
101                 return float(week_str)
102             except:
103                 return np.nan
104     else:
105         try:
106             return float(week_str)
107         except:
108             return np.nan
109
110 def load_and_preprocess_data(file_path):
111     """加载和预处理数据"""
112     print("正在加载数据...")
113     data = pd.read_excel(file_path)
114     print(f"原始数据形状: {data.shape}")
115
116     # 转换孕周格式
117     data['检测孕周数值'] = data['检测孕周'].apply(
convert_gestational_week)
118
119     # 数据清洗

```

```

120     data_clean = data.dropna(subset=['检测孕周数值', '孕妇BMI'
121     , 'Y染色体浓度', '年龄'])
122
122     # 筛选男胎数据
123     male_data = data_clean[data_clean['Y染色体浓度'] > 0]
124     male_data = male_data[(male_data['检测孕周数值'] >= 10) &
125     (male_data['检测孕周数值'] <= 25)]
126     male_data = male_data[(male_data['Y染色体浓度'] > 0.1) & (
127     male_data['Y染色体浓度'] < 20)]
128
127     print(f"清洗后男胎数据形状: {male_data.shape}")
128     return male_data
129
130 def advanced_feature_engineering(data):
131     """高级特征工程"""
132     df = data.copy()
133
134     # 基础交互特征
135     df['孕周_BMI'] = df['检测孕周数值'] * df['孕妇BMI']
136     df['孕周_年龄'] = df['检测孕周数值'] * df['年龄']
137     df['BMI_年龄'] = df['孕妇BMI'] * df['年龄']
138
139     # 多项式特征
140     df['孕周平方'] = df['检测孕周数值'] ** 2
141     df['BMI平方'] = df['孕妇BMI'] ** 2
142     df['年龄平方'] = df['年龄'] ** 2
143
144     # 对数变换
145     df['log_孕周'] = np.log1p(df['检测孕周数值'])
146     df['log_BMI'] = np.log1p(df['孕妇BMI'])
147
148     # 比率特征
149     df['孕周_BMI比率'] = df['检测孕周数值'] / df['孕妇BMI']
150
151     print("特征工程完成")

```

```

152     return df
153
154 def optimized_random_forest(data):
155     """优化随机森林"""
156     print("\n=== 优化随机森林 ===")
157
158     features = ['检测孕周数值', '孕妇BMI', '年龄', '孕周_BMI',
159                '孕周_年龄',
160                '孕周平方', 'BMI平方', 'log_孕周', 'log_BMI']
161
162     X = data[[col for col in features if col in data.columns
163 ]].copy()
164     y = data['Y染色体浓度']
165
166     # 处理缺失值
167     for col in X.columns:
168         if X[col].isnull().any():
169             X[col].fillna(X[col].median(), inplace=True)
170
171     valid_mask = X.notna().all(axis=1) & y.notna()
172     X = X[valid_mask]
173     y = y[valid_mask]
174
175     # 参数优化
176     param_grid = {
177         'n_estimators': [100, 200],
178         'max_depth': [5, 10, None],
179         'min_samples_split': [2, 5]
180     }
181
182     rf = RandomForestRegressor(random_state=42)
183     grid_search = GridSearchCV(rf, param_grid, cv=3, scoring='
r2', n_jobs=-1)
184     grid_search.fit(X, y)
185

```

```

184     best_rf = grid_search.best_estimator_
185     y_pred = best_rf.predict(X)
186     r2 = r2_score(y, y_pred)
187
188     print(f"最佳参数: {grid_search.best_params_}")
189     print(f"优化随机森林 R²: {r2:.4f}")
190
191     # 特征重要性
192     feature_importance = pd.DataFrame({
193         'feature': X.columns,
194         'importance': best_rf.feature_importances_
195     }).sort_values('importance', ascending=False)
196
197     return best_rf, r2, feature_importance
198
199 def optimized_gradient_boosting(data):
200     """优化梯度提升"""
201     print("\n=== 优化梯度提升 ===")
202
203     features = ['检测孕周数值', '孕妇BMI', '年龄', '孕周_BMI',
204                '孕周平方', 'BMI平方']
205
206     X = data[[col for col in features if col in data.columns
207                ]].copy()
208     y = data['Y染色体浓度']
209
210     for col in X.columns:
211         if X[col].isnull().any():
212             X[col].fillna(X[col].median(), inplace=True)
213
214     valid_mask = X.notna().all(axis=1) & y.notna()
215     X = X[valid_mask]
216     y = y[valid_mask]
217
218     # 参数优化

```

```

217     gb = GradientBoostingRegressor(random_state=42)
218     param_grid = {
219         'n_estimators': [100, 200],
220         'learning_rate': [0.05, 0.1],
221         'max_depth': [3, 4]
222     }
223
224     grid_search = GridSearchCV(gb, param_grid, cv=3, scoring='
r2', n_jobs=-1)
225     grid_search.fit(X, y)
226
227     best_gb = grid_search.best_estimator_
228     y_pred = best_gb.predict(X)
229     r2 = r2_score(y, y_pred)
230
231     print(f"最佳参数: {grid_search.best_params}")
232     print(f"优化梯度提升 R2: {r2:.4f}")
233
234     return best_gb, r2
235
236 def feature_selection_analysis(data):
237     """特征选择分析"""
238     print("\n=== 特征选择分析 ===")
239
240     features = ['检测孕周数值', '孕妇BMI', '年龄', '孕周_BMI',
241                '孕周_年龄',
242                '孕周平方', 'BMI平方', 'log_孕周', 'log_BMI']
243
244     available_features = [col for col in features if col in
data.columns]
245
246     X = data[available_features].copy()
247     y = data['Y染色体浓度']
248
249     for col in available_features:

```

```

249         if X[col].isnull().any():
250             X[col].fillna(X[col].median(), inplace=True)
251
252     valid_mask = X.notna().all(axis=1) & y.notna()
253     X = X[valid_mask]
254     y = y[valid_mask]
255
256     # 特征选择
257     selector = SelectFromModel(
258         RandomForestRegressor(n_estimators=100, random_state
=42),
259         threshold='median'
260     )
261
262     selector.fit(X, y)
263     selected_features = X.columns[selector.get_support()].
tolist()
264
265     print(f"选择的特征: {selected_features}")
266
267     # 使用选择的特征重新训练
268     X_selected = X[selected_features]
269     model = RandomForestRegressor(n_estimators=200,
random_state=42)
270     model.fit(X_selected, y)
271
272     y_pred = model.predict(X_selected)
273     r2 = r2_score(y, y_pred)
274
275     print(f"特征选择后模型 R2: {r2:.4f}")
276
277     return model, r2, selected_features
278
279 # ===== 可视化函数 =====
280 def create_correlation_heatmap(data):

```

```

281     """创建变量相关性热力图"""
282     print("生成变量相关性热力图...")
283
284     numeric_cols = ['检测孕周数值', '孕妇BMI', '年龄', 'Y染色体浓度', '孕周_BMI', '孕周_年龄', '孕周平方', 'BMI平方']
285     numeric_cols = [col for col in numeric_cols if col in data.columns]
286
287     corr_matrix = data[numeric_cols].corr()
288
289     plt.figure(figsize=(12, 10))
290     sns.heatmap(corr_matrix, annot=True, cmap='RdBu_r', center=0,
291                 square=True, fmt='.2f', cbar_kws={"shrink":
292                 .8})
293     plt.title('变量相关性热力图', fontsize=16, fontweight='bold')
294     plt.tight_layout()
295     plt.savefig('2_correlation_heatmap.png', dpi=300,
296                 bbox_inches='tight')
297     plt.show()
298
299 def create_feature_importance_plot(feature_importance):
300     """创建特征重要性排名图"""
301     print("生成特征重要性排名图...")
302
303     plt.figure(figsize=(12, 8))
304     top_features = feature_importance.head(8)
305     colors = plt.cm.viridis(np.linspace(0, 1, len(top_features)))
306
307     bars = plt.barh(range(len(top_features)), top_features['importance'], color=colors)
308     plt.yticks(range(len(top_features)), top_features['feature'])

```

```

307     plt.xlabel('特征重要性')
308     plt.title('Top 特征重要性排名', fontsize=16, fontweight='
bold')
309
310     for i, bar in enumerate(bars):
311         width = bar.get_width()
312         plt.text(width + 0.001, bar.get_y() + bar.get_height()
/2,
313                 f'{width:.4f}', ha='left', va='center')
314
315     plt.gca().invert_yaxis()
316     plt.tight_layout()
317     plt.savefig('3_feature_importance.png', dpi=300,
bbox_inches='tight')
318     plt.show()
319
320 def create_model_comparison_plot(results):
321     """创建模型性能对比图"""
322     print("生成模型性能对比图...")
323
324     plt.figure(figsize=(10, 6))
325     models = list(results.keys())
326     r2_scores = list(results.values())
327
328     colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4']
329     bars = plt.bar(models, r2_scores, color=colors[:len(models
)])
330
331     plt.xlabel('模型')
332     plt.ylabel('R2 分数')
333     plt.title('不同模型性能对比', fontsize=16, fontweight='
bold')
334     plt.xticks(rotation=45)
335     plt.ylim(0, max(r2_scores) * 1.1)
336

```



```

337     for bar, score in zip(bars, r2_scores):
338         height = bar.get_height()
339         plt.text(bar.get_x() + bar.get_width()/2., height +
0.01,
340                 f'{score:.4f}', ha='center', va='bottom')
341
342     plt.tight_layout()
343     plt.savefig('4_model_comparison.png', dpi=300, bbox_inches
='tight')
344     plt.show()
345
346 # ===== 主函数 =====
347 def main_simplified_analysis():
348     """简化分析流程"""
349     file_path = r"D:\数学建模\省赛\C题\附件.xlsx"
350
351     try:
352         print("开始简化分析流程...")
353         print("="*50)
354
355         # 1. 加载数据
356         data = load_and_preprocess_data(file_path)
357
358         # 2. 特征工程
359         data = advanced_feature_engineering(data)
360         print(f"最终数据形状: {data.shape}")
361
362         # 3. 模型训练
363         rf_model, rf_r2, feature_importance =
optimized_random_forest(data)
364         gb_model, gb_r2 = optimized_gradient_boosting(data)
365         fs_model, fs_r2, selected_features =
feature_selection_analysis(data)
366
367         # 4. 结果汇总

```

```

368     results = {
369         'RandomForest': rf_r2,
370         'GradientBoosting': gb_r2,
371         'FeatureSelection': fs_r2
372     }
373
374     # 5. 生成可视化图表
375     print("\n正在生成可视化图表...")
376     create_correlation_heatmap(data)
377     create_feature_importance_plot(feature_importance)
378     create_model_comparison_plot(results)
379
380     # 6. 最终结果
381     print("\n" + "="*50)
382     print("最终分析结果")
383     print("="*50)
384
385     for model, r2 in results.items():
386         print(f"{model}: {r2:.4f}")
387
388     best_model = max(results.items(), key=lambda x: x[1])
389     print(f"\n□□ 最佳模型: {best_model[0]} (R² = {
best_model[1]:.4f})")
390
391     print(f"\n□□ 最重要的特征:")
392     top_3 = feature_importance.head(3)
393     for _, row in top_3.iterrows():
394         print(f"    {row['feature']}: {row['importance']:.4f
}")
395
396     print(f"\n□ 分析完成! 已生成3张核心图表: ")
397     print("1. 2_correlation_heatmap.png - 变量相关性热力图
")
398     print("2. 3_feature_importance.png - 特征重要性排名图"
)

```

```

399         print("3. 4_model_comparison.png - 模型性能对比图")
400
401     return {
402         'data': data,
403         'results': results,
404         'feature_importance': feature_importance,
405         'selected_features': selected_features
406     }
407
408     except Exception as e:
409         print(f"分析过程中出现错误: {e}")
410         import traceback
411         traceback.print_exc()
412         return None
413
414 # 运行简化分析
415 if __name__ == "__main__":
416     results = main_simplified_analysis()

```

q2.py

```

1  import pandas as pd
2  import re
3  from sklearn.preprocessing import PolynomialFeatures
4  from sklearn.linear_model import LinearRegression
5  import matplotlib.pyplot as plt
6  import numpy as np
7  import os # 导入os模块, 用于检查文件是否存在
8  from sklearn.cluster import KMeans
9  from copy import deepcopy
10
11 # 设置中文字体, 确保乱码问题解决 - 增大字体大小
12 plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei",
13     , "Heiti TC", "sans-serif"]
14 plt.rcParams["axes.unicode_minus"] = False # 解决负号显示问题
15 plt.rcParams['font.size'] = 14 # 增大基础字体大小

```

```

15 plt.rcParams['axes.titlesize'] = 18 # 增大标题字体大小
16 plt.rcParams['axes.labelsize'] = 16 # 增大坐标轴标签字体大小
17 plt.rcParams['xtick.labelsize'] = 14 # 增大x轴刻度字体大小
18 plt.rcParams['ytick.labelsize'] = 14 # 增大y轴刻度字体大小
19 plt.rcParams['legend.fontsize'] = 14 # 增大图例字体大小
20
21 # 1. 数据读取与格式转换
22 # 请将文件路径修改为你实际的Excel文件路径
23 file_path = "D:\\数学建模\\省赛\\C题\\附件.xlsx"
24
25 # 检查文件是否存在
26 if not os.path.exists(file_path):
27     raise FileNotFoundError(f"文件未找到: {file_path}, 请检查
    路径是否正确! ")
28
29 df = pd.read_excel(file_path)
30
31 required_cols = ["孕妇BMI", "检测孕周", "Y染色体浓度"]
32 for col in required_cols:
33     if col not in df.columns:
34         raise ValueError(f"Excel中缺少必要列: {col}, 请核对表
    头! ")
35
36 # 孕周转换函数
37 def convert_gestational_week(week_str):
38     try:
39         match = re.match(r'^s*(\d+)\s*w\s*\+\s*(\d+)\s*$',
    str(week_str).strip())
40         if match and 0 <= int(match.group(2)) <= 6:
41             return round(int(match.group(1)) + int(match.group
    (2))/7, 3)
42         return None
43     except Exception as e:
44         print(f"孕周转换错误 (输入: {week_str}) : {e}")
45         return None

```

```

46
47 # 数据转换与清洗
48 df["孕妇BMI"] = pd.to_numeric(df["孕妇BMI"], errors="coerce")
49 # 注意这里的列名是"检测孕周_小数", 与原始代码一致
50 df["检测孕周_小数"] = df["检测孕周"].apply(
    convert_gestational_week)
51 df["Y染色体浓度"] = pd.to_numeric(df["Y染色体浓度"], errors="
    coerce")
52
53 valid_df = df[
54     (df["孕妇BMI"].notna()) &
55     (df["孕妇BMI"] > 10) & (df["孕妇BMI"] < 50) &
56     (df["检测孕周_小数"].notna()) &
57     (df["检测孕周_小数"] >= 8) & (df["检测孕周_小数"] <= 40) &
58     (df["Y染色体浓度"].notna())
59 ].copy()
60
61 total_valid = len(valid_df)
62 if total_valid == 0:
63     raise ValueError("无有效数据! 请检查数据格式或范围是否正确
        ")
64
65 print(f"有效样本数: {total_valid}")
66
67 # 2. 多项式回归模型拟合
68 # 特征列名是["孕妇BMI", "检测孕周_小数"]
69 X = valid_df[["孕妇BMI", "检测孕周_小数"]]
70 y = valid_df["Y染色体浓度"]
71
72 poly = PolynomialFeatures(degree=2, include_bias=False)
73 X_poly = poly.fit_transform(X)
74
75 # 关键修正: 传入与特征列名完全一致的名称
76 feature_names = poly.get_feature_names_out(["孕妇BMI", "检测孕
    周_小数"])

```

```

77
78 model = LinearRegression()
79 model.fit(X_poly, y)
80
81 r2 = model.score(X_poly, y)
82 print(f"R2值: {r2:.4f}")
83
84 # 输出回归方程
85 print("回归方程: ")
86 print(f"Y染色体浓度 = {model.intercept_:.6f}")
87 for name, coef in zip(feature_names, model.coef_):
88     print(f"    + {coef:.6f} × {name}")
89
90 # 3. 可视化 - 优化字体大小和可读性
91 valid_df["预测Y浓度"] = model.predict(X_poly)
92
93 # 创建更大的图形，确保所有元素都能清晰显示
94 fig, ax = plt.subplots(figsize=(14, 10)) # 增大图形尺寸
95
96 scatter = ax.scatter(
97     valid_df["检测孕周_小数"],
98     valid_df["Y染色体浓度"],
99     c=valid_df["孕妇BMI"],
100     cmap="viridis",
101     alpha=0.7, # 增加透明度，使点更清晰
102     s=80,      # 增大点的大小
103     edgecolors="black",
104     linewidth=0.5, # 减小边缘线宽
105     label="实际值"
106 )
107
108 ax.scatter(
109     valid_df["检测孕周_小数"],
110     valid_df["预测Y浓度"],
111     c="red",

```

```

112     alpha=0.7,
113     s=50,          # 增大预测点的大小
114     marker="x",
115     linewidth=2,   # 增大线宽
116     label="预测值"
117 )
118
119 # 添加颜色条并增大字体
120 cbar = plt.colorbar(scatter, pad=0.05) # 调整颜色条位置
121 cbar.set_label("孕妇BMI", fontsize=16) # 增大颜色条标签字体
122 cbar.ax.tick_params(labelsize=14) # 增大颜色条刻度字体
123
124 # 设置坐标轴标签和标题
125 ax.set_xlabel("检测孕周 (周)", fontsize=16, labelpad=10) # 增大
    标签字体并增加间距
126 ax.set_ylabel("Y染色体浓度", fontsize=16, labelpad=10) # 增大
    标签字体并增加间距
127
128 # 设置标题
129 ax.set_title(f"多项式回归模型拟合 ( $R^2={r2:.4f}$ )", fontsize=20,
    pad=20) # 增大标题字体
130
131 # 设置网格
132 ax.grid(alpha=0.3)
133
134 # 添加图例并调整位置
135 legend = ax.legend(loc='best', fontsize=14, framealpha=0.9)
136 legend.get_frame().set_edgecolor('black')
137
138 # 调整布局以确保所有元素都能完整显示
139 plt.tight_layout()
140
141 # 保存高质量图像
142 plt.savefig('polynomial_regression_fit.png', dpi=300,
    bbox_inches='tight')

```

```

143
144 # 显示图形
145 plt.show()
146
147
148
149 # 解决中文乱码 - 增大字体大小
150 plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei"
    , "Heiti TC"]
151 plt.rcParams["axes.unicode_minus"] = False
152 plt.rcParams['font.size'] = 14 # 增大基础字体大小
153 plt.rcParams['axes.titlesize'] = 18 # 增大标题字体大小
154 plt.rcParams['axes.labelsize'] = 16 # 增大坐标轴标签字体大小
155 plt.rcParams['xtick.labelsize'] = 14 # 增大x轴刻度字体大小
156 plt.rcParams['ytick.labelsize'] = 14 # 增大y轴刻度字体大小
157 plt.rcParams['legend.fontsize'] = 14 # 增大图例字体大小
158
159 # 定义颜色方案
160 colors = ['#4169E1', '#9932CC', '#FFA500', '#32CD32', '#FF4500
    '] # 蓝、紫、橙、绿、红
161
162 # 定义孕周转换函数：将"11w+6"格式转换为周数（保留2位小数）
163 def convert_gestational_week(week_str):
164     try:
165         match = re.match(r'^\s*(\d+)\s*[wW]\s*\+\s*(\d+)\s*$',
166             str(week_str))
167         if match:
168             weeks = int(match.group(1))
169             days = int(match.group(2))
170             if 0 <= days <= 6:
171                 return round(weeks + days / 7, 2)
172             return None
173         except Exception as e:
174             print(f"孕周转换错误（输入值：{week_str}）：{e}")
175             return None

```



```

175
176 # 读取原始数据
177 file_path = r"D:\数学建模\省赛\C题\附件.xlsx"
178 df_original = pd.read_excel(file_path)
179
180 # 转换孕周为数值型
181 df_original['检测孕周_数值'] = df_original['检测孕周'].apply(
    convert_gestational_week)
182
183 # 原始数据清洗（与原代码一致）
184 df_original = df_original.dropna(subset=['孕妇BMI', '检测孕周_
    数值', 'Y染色体浓度'])
185 df_original = df_original[(df_original['孕妇BMI'] > 10) & (
    df_original['孕妇BMI'] < 50) &
186                             (df_original['检测孕周_数值'] >= 8)
    & (df_original['检测孕周_数值'] <= 40)]
187
188 if len(df_original) == 0:
189     raise ValueError("没有有效数据！请检查数据格式或清洗条件")
190
191 # 定义误差幅度（如5%、10%、15%的相对误差）
192 error_levels = [0.05, 0.1, 0.15]
193
194 # 存储不同误差下的聚类结果
195 all_cluster_results = {}
196
197 # 对每个误差幅度进行测试
198 for error_pct in error_levels:
199     print(f"\n=== 模拟误差幅度：{error_pct*100}% ===")
200     # 深拷贝原始数据，避免修改原始数据
201     df = deepcopy(df_original)
202
203     # 注入随机误差：对孕妇BMI和检测孕周_数值添加正态分布噪声
204     # 误差幅度为特征值的 error_pct 倍，标准差由 error_pct * 特
    征值 计算

```

```

205     df['孕妇BMI'] = df['孕妇BMI'] + np.random.normal(0, df['孕
206     妇BMI'] * error_pct, size=len(df))
207
208     df['检测孕周_数值'] = df['检测孕周_数值'] + np.random.
209     normal(0, df['检测孕周_数值'] * error_pct, size=len(df))
210
211     # 提取聚类变量（仅以BMI为主要聚类变量，与原代码一致）
212     X = df[['孕妇BMI']].values
213
214     # 聚类（分为3类，与原代码一致）
215     n_clusters = 3
216     kmeans = KMeans(n_clusters=n_clusters, random_state=42)
217     df['cluster'] = kmeans.fit_predict(X)
218
219     cluster_info = []
220     for cluster_id in range(n_clusters):
221         cluster_data = df[df['cluster'] == cluster_id].copy()
222         bmi_min = round(cluster_data['孕妇BMI'].min(), 2)
223         bmi_max = round(cluster_data['孕妇BMI'].max(), 2)
224
225         valid_data = cluster_data[cluster_data['Y染色体浓度']
226         >= 0.04]
227
228         if len(valid_data) == 0:
229             best_nipt_week = None
230         else:
231             best_nipt_week = round(valid_data['检测孕周_数值']
232             .quantile(0.9), 2)
233
234         cluster_info.append({
235             "cluster_id": cluster_id,
236             "bmi_range": (bmi_min, bmi_max),
237             "best_nipt_week": best_nipt_week
238         })
239
240     # 存储当前误差下的聚类信息
241     all_cluster_results[error_pct] = cluster_info

```

```

236
237     # 打印当前误差下的聚类结果
238     for info in cluster_info:
239         print(f"聚类{info['cluster_id']} | BMI区间: [{info['bmi_range'][0]}, {info['bmi_range'][1]}] | 最佳检测时间: {info['best_nipt_week']}周")
240
241 # 可视化不同误差下的聚类对比（以BMI区间和最佳孕周为例）
242 # 创建更大的图形，确保所有元素都能清晰显示
243 fig = plt.figure(figsize=(18, 10)) # 增加高度以容纳底部图例
244
245 # 设置主标题
246 plt.suptitle('不同检测误差下的BMI聚类与最佳NIPT时点变化',
247             fontsize=20, fontweight='bold', y=0.95)
248
249 # 绘制每个误差水平的子图
250 for i, error_pct in enumerate(error_levels):
251     ax = plt.subplot(1, len(error_levels), i+1)
252     cluster_info = all_cluster_results[error_pct]
253
254     for cluster_id in range(n_clusters):
255         # 模拟带误差的数据用于绘图（与误差注入逻辑一致）
256         df_error = deepcopy(df_original)
257         df_error['孕妇BMI'] = df_error['孕妇BMI'] + np.random.normal(0, df_error['孕妇BMI'] * error_pct, size=len(df_error))
258         df_error['检测孕周_数值'] = df_error['检测孕周_数值'] + np.random.normal(0, df_error['检测孕周_数值'] * error_pct, size=len(df_error))
259         df_error['cluster'] = KMeans(n_clusters=n_clusters, random_state=42).fit_predict(df_error[['孕妇BMI']].values)
260
261         cluster_data = df_error[df_error['cluster'] == cluster_id]
262         scatter = ax.scatter(

```

```

262         cluster_data['孕妇BMI'],
263         cluster_data['检测孕周_数值'],
264         color=colors[cluster_id], # 使用预定义的颜色
265         alpha=0.7,
266         s=60, # 增大点的大小
267         label=f'聚类{cluster_id}' if i == 0 else "" # 只
在第一个子图设置标签
268     )
269
270     best_week = cluster_info[cluster_id]["best_nipt_week"]
271     if best_week is not None:
272         ax.axhline(y=best_week, color=colors[cluster_id],
linestyle='--', linewidth=2.0,
273                 label=f'聚类{cluster_id}最佳检测周' if i
== 0 else "") # 只在第一个子图设置标签
274
275     ax.set_xlabel('孕妇BMI', fontsize=16, labelpad=10) # 增大
标签字体并增加间距
276     ax.set_ylabel('检测孕周（周）', fontsize=16, labelpad=10)
# 增大标签字体并增加间距
277
278     ax.set_title(f'误差幅度 {error_pct*100}%', fontsize=18,
pad=15) # 增大子标题字体并增加间距
279
280     ax.grid(alpha=0.3)
281
282     # 增大刻度标签字体
283     ax.tick_params(axis='both', which='major', labelsize=14)
284
285 # 创建统一的图例并放在图形底部
286 handles, labels = [], []
287 for cluster_id in range(n_clusters):
288     # 添加聚类点的图例
289     handles.append(plt.Line2D([0], [0], marker='o', color='w',
markerfacecolor=colors[cluster_id],

```

```

290             markersize=10, label=f'聚类{
cluster_id}')))
291     # 添加最佳检测周的图例
292     handles.append(plt.Line2D([0], [0], color=colors[
cluster_id], linestyle='--', linewidth=2,
293             label=f'聚类{cluster_id}最佳检测
周'))
294
295 # 将图例放在图形底部中央
296 fig.legend(handles, [h.get_label() for h in handles],
297         loc='lower center',
298         ncol=n_clusters, # 每行显示n_clusters个图例项
299         fontsize=14,
300         framealpha=0.9,
301         bbox_to_anchor=(0.5, 0.02)) # 调整图例位置
302
303 # 调整布局以确保所有元素都能完整显示
304 plt.tight_layout(rect=[0, 0.08, 1, 0.95]) # 为底部图例和顶部
标题留出空间
305
306 # 保存高质量图像
307 plt.savefig('error_analysis_clusters.png', dpi=300,
bbox_inches='tight')
308
309 # 显示图形
310 plt.show()
311
312 # 量化分析：计算各聚类BMI区间的波动范围
313 print("\n==== 各误差下聚类BMI区间波动分析 ====")
314 for cluster_id in range(n_clusters):
315     bmi_mins = []
316     bmi_maxs = []
317     best_weeks = []
318     for error_pct in error_levels:
319         info = all_cluster_results[error_pct][cluster_id]

```

```

320         bmi_mins.append(info['bmi_range'][0])
321         bmi_maxs.append(info['bmi_range'][1])
322         best_weeks.append(info['best_nipt_week'] if info['
best_nipt_week'] is not None else np.nan)
323
324     # 计算BMI区间的波动范围
325     bmi_min_range = (min(bmi_mins), max(bmi_mins))
326     bmi_max_range = (min(bmi_maxs), max(bmi_maxs))
327     best_week_range = (np.nanmin(best_weeks), np.nanmax(
best_weeks)) if not all(np.isnan(best_weeks)) else (None,
None)
328
329     print(f"聚类{cluster_id}:")
330     print(f"    BMI最小值波动: {bmi_min_range[0]:.2f} ~ {
bmi_min_range[1]:.2f}")
331     print(f"    BMI最大值波动: {bmi_max_range[0]:.2f} ~ {
bmi_max_range[1]:.2f}")
332     if best_week_range[0] is not None:
333         print(f"    最佳检测周波动: {best_week_range[0]:.2f} ~ {
best_week_range[1]:.2f}")
334     else:
335         print("    最佳检测周: 无有效数据")

```

q3.py

```

1  import pandas as pd
2  import re
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from sklearn.preprocessing import MinMaxScaler
6  from sklearn.cluster import KMeans
7  from sklearn.metrics import silhouette_score
8  from lifelines import KaplanMeierFitter
9  from lifelines.statistics import logrank_test
10 from scipy import stats
11 import warnings

```

```

12 warnings.filterwarnings('ignore')
13
14 # ----- 第二步：基础设置（避免中文乱码）
    -----
15 # 增大字体设置
16 plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei'
    , 'Heiti TC']
17 plt.rcParams['axes.unicode_minus'] = False
18 plt.rcParams['font.size'] = 12 # 增大基础字体大小
19 plt.rcParams['axes.titlesize'] = 16 # 增大标题字体大小
20 plt.rcParams['axes.labelsize'] = 14 # 增大坐标轴标签字体大小
21 plt.rcParams['xtick.labelsize'] = 12 # 增大x轴刻度字体大小
22 plt.rcParams['ytick.labelsize'] = 12 # 增大y轴刻度字体大小
23 plt.rcParams['legend.fontsize'] = 12 # 增大图例字体大小
24
25 # ----- 第三步：数据加载与孕周转换
    -----
26 file_path = "D:/数学建模/省赛/C题/附件.xlsx"
27 data = pd.read_excel(file_path)
28
29 def convert_gestational_week(week_str):
30     try:
31         if pd.isna(week_str) or not isinstance(week_str, str):
32             return None
33         pattern = r'^s*(\d+)\s*[wW]\s*\+?\s*(\d*)\s*$'
34         match = re.match(pattern, week_str.strip())
35         if match:
36             weeks = int(match.group(1))
37             days = int(match.group(2)) if match.group(2) else
0
38             if 0 <= days <= 6:
39                 return round(weeks + days / 7, 2)
40             return round(float(week_str), 2)
41     except Exception as e:
42         print(f"孕周转换错误（输入值：{week_str}）：{str(e)}")

```

```

43         return None
44
45 data['检测孕周_数值'] = data['检测孕周'].apply(
    convert_gestational_week)
46 data['time'] = data['检测孕周_数值']
47 data['event'] = (data['Y染色体浓度'] >= 0.04).astype(int)
48
49 # ----- 第四步：聚类分析
    -----
50 features = ['孕妇BMI', '年龄', '身高', '体重']
51 data_clean = data.dropna(subset=features + ['time', 'event'])
52 data_clean = data_clean[
53     (data_clean['孕妇BMI'] > 10) & (data_clean['孕妇BMI'] <
54     50) &
55     (data_clean['年龄'] >= 16) & (data_clean['年龄'] <= 50) &
56     (data_clean['time'] >= 8) & (data_clean['time'] <= 40)
57 ]
58 print(f"有效样本数: {len(data_clean)}")
59
60 scaler = MinMaxScaler()
61 data_scaled = scaler.fit_transform(data_clean[features])
62
63 silhouette_scores = []
64 k_range = range(2, 11)
65 for k in k_range:
66     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
67     labels = kmeans.fit_predict(data_scaled)
68     if k > 1:
69         silhouette_scores.append(silhouette_score(data_scaled,
70         labels))
71
72 best_k = k_range[np.argmax(silhouette_scores)]
73 print(f"最佳聚类数: {best_k} (轮廓系数: {max(silhouette_scores
74     ):.4f}) ")
75
76 kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)

```



```

73 data_clean['聚类标签'] = kmeans.fit_predict(data_scaled)
74
75 # ----- 第五步：绘制生存曲线 + 提取交点
76     -----
77 kmf = KaplanMeierFitter()
78
79 # 为生存曲线图单独设置更大的字体
80 plt.rcParams.update({
81     'font.size': 14, # 增大基础字体大小
82     'axes.titlesize': 18, # 增大标题字体大小
83     'axes.labelsize': 16, # 增大坐标轴标签字体大小
84     'xtick.labelsize': 14, # 增大x轴刻度字体大小
85     'ytick.labelsize': 14, # 增大y轴刻度字体大小
86     'legend.fontsize': 14 # 增大图例字体大小
87 })
88
89 plt.figure(figsize=(14, 9)) # 稍微增大图形尺寸
90 colors = plt.cm.viridis(np.linspace(0, 1, best_k))
91 intersection_x = {}
92
93 for i, cluster in enumerate(range(best_k)):
94     cluster_data = data_clean[data_clean['聚类标签'] ==
95                             cluster]
96     T = cluster_data['time']
97     E = cluster_data['event']
98
99     kmf.fit(T, event_observed=E, label=f'聚类{cluster} (样本
100 数: {len(cluster_data)}) ')
101     kmf.plot_survival_function(ci_show=True, color=colors[i],
102                             linewidth=2, drawstyle='steps-post')
103
104     survival_function = kmf.survival_function_
105     times = survival_function.index.values
106     probs = survival_function.iloc[:, 0].values

```

```

104     if (probs <= 0.2).any():
105         idx = np.argmax(probs <= 0.2)
106         if idx == 0:
107             x = times[idx]
108         else:
109             t1, p1 = times[idx-1], probs[idx-1]
110             t2, p2 = times[idx], probs[idx]
111             x = t1 + (0.2 - p1) * (t2 - t1) / (p2 - p1)
112             intersection_x[f'聚类{cluster}'] = round(x, 2)
113     else:
114         intersection_x[f'聚类{cluster}'] = '无足够数据'
115
116 plt.axhline(y=0.2, color='red', linestyle='--', alpha=0.8,
117             label='80%达标线（未达标概率=0.2）', linewidth=2)
118 plt.xlabel('检测孕周（周）', fontsize=16, labelpad=12)
119 plt.ylabel('未达标概率（生存概率）', fontsize=16, labelpad=12)
120 plt.title('不同聚类群体的Y染色体浓度达标生存曲线（Kaplan-Meier
121           ）', fontsize=18, pad=20)
122 plt.grid(alpha=0.3)
123 plt.legend(loc='lower left', fontsize=14)
124 plt.tight_layout()
125 plt.savefig('生存曲线.png', dpi=300, bbox_inches='tight')
126 plt.show()
127
128 # 绘制完生存曲线后，恢复全局字体设置
129 plt.rcParams.update({
130     'font.size': 12,
131     'axes.titlesize': 16,
132     'axes.labelsize': 14,
133     'xtick.labelsize': 12,
134     'ytick.labelsize': 12,
135     'legend.fontsize': 12
136 })
137
138 # ----- 第六步：误差分析模块（修复版本）

```

```

-----
137 def perform_error_analysis(data_clean, intersection_x, best_k)
    :
138     """执行全面的误差分析"""
139     print("\n" + "="*60)
140     print("误差分析报告")
141     print("="*60)
142
143     # 1. 聚类质量评估
144     print("\n1. 聚类质量评估:")
145     print(f"轮廓系数: {max(silhouette_scores):.4f}")
146     if max(silhouette_scores) > 0.7:
147         print("□ 聚类质量优秀")
148     elif max(silhouette_scores) > 0.5:
149         print("□ 聚类质量一般")
150     else:
151         print("□ 聚类质量较差")
152
153     # 2. 样本量分析
154     print("\n2. 样本量分析:")
155     for cluster in range(best_k):
156         n_samples = len(data_clean[data_clean['聚类标签'] ==
cluster])
157         print(f"聚类{cluster}: {n_samples} 个样本")
158         if n_samples < 30:
159             print(f"□ 样本量较少, 可能影响统计显著性")
160
161     # 3. 生存曲线显著性检验
162     print("\n3. 生存曲线差异显著性检验 (Log-Rank Test):")
163     if best_k >= 2:
164         # 对所有聚类组合进行两两比较
165         for i in range(best_k):
166             for j in range(i+1, best_k):
167                 group_i = data_clean[data_clean['聚类标签'] ==
i]

```

```

168         group_j = data_clean[data_clean['聚类标签'] ==
169 j]
170
171         try:
172             result = logrank_test(
173                 group_i['time'], group_j['time'],
174                 event_observed_A=group_i['event'],
175                 event_observed_B=group_j['event']
176             )
177
178             significance = "显著不同" if result.
179 p_value < 0.05 else "无显著差异"
180             print(f"  聚类{i} vs 聚类{j}: p值={result.
181 p_value:.4f} ({significance})")
182         except Exception as e:
183             print(f"  聚类{i} vs 聚类{j}: 检验失败 - {
184 str(e)}")
185
186 # 4. 置信区间分析（修复版本）
187 print("\n4. 置信区间分析:")
188 for cluster in range(best_k):
189     cluster_data = data_clean[data_clean['聚类标签'] ==
190 cluster]
191     T = cluster_data['time']
192     E = cluster_data['event']
193
194     if len(T) > 0:
195         kmf.fit(T, event_observed=E)
196
197         # 修复：使用 timeline 而不是 timeline_
198         if hasattr(kmf, 'timeline'):
199             timeline = kmf.timeline
200         elif hasattr(kmf, 'timeline_'):
201             timeline = kmf.timeline_
202         else:

```

```

198         print(f"聚类{cluster}: 无法获取时间线")
199         continue
200
201     if len(timeline) > 0:
202         last_time = timeline[-1]
203
204         # 获取置信区间
205         if hasattr(kmf, 'confidence_interval_'):
206             ci_data = kmf.confidence_interval_
207         elif hasattr(kmf, '
confidence_interval_survival_function_'):
208             ci_data = kmf.
confidence_interval_survival_function_
209         else:
210             print(f"聚类{cluster}: 无法获取置信区间")
211             continue
212
213         # 找到最后一个时间点的置信区间
214         if last_time in ci_data.index:
215             ci = ci_data.loc[last_time]
216             if len(ci) >= 2:
217                 print(f"聚类{cluster} 最终时间点置信区
间: [{ci.iloc[0]:.3f}, {ci.iloc[1]:.3f}]")
218                 ci_width = ci.iloc[1] - ci.iloc[0]
219                 if ci_width > 0.2:
220                     print(f" □ 置信区间较宽 ({
ci_width:.3f}), 结果不确定性较大")
221                 else:
222                     print(f"聚类{cluster}: 置信区间数据不
完整")
223             else:
224                 print(f"聚类{cluster}: 无法找到最终时间点
的置信区间")
225         else:
226             print(f"聚类{cluster}: 时间线为空")

```

```

227         else:
228             print(f"聚类{cluster}: 无有效数据")
229
230     # 5. 达标时间误差估计（优化版本）
231     print("\n5. 80%达标时间误差估计:")
232     for cluster, x_value in intersection_x.items():
233         if isinstance(x_value, (int, float)):
234             cluster_idx = int(cluster.replace('聚类', ''))
235             cluster_data = data_clean[data_clean['聚类标签']
== cluster_idx]
236
237             if len(cluster_data) < 10:
238                 print(f"{cluster}: 样本量不足，无法进行误差估
计")
239                 continue
240
241             # 使用Bootstrap方法估计误差
242             n_bootstrap = 500 # 减少重采样次数以提高速度
243             bootstrap_estimates = []
244
245             for _ in range(n_bootstrap):
246                 try:
247                     # 重采样
248                     bootstrap_sample = cluster_data.sample(n=
len(cluster_data), replace=True)
249                     T = bootstrap_sample['time']
250                     E = bootstrap_sample['event']
251
252                     if len(T) == 0:
253                         continue
254
255                     # 拟合KM曲线
256                     kmf_bootstrap = KaplanMeierFitter()
257                     kmf_bootstrap.fit(T, event_observed=E)
258

```

```

259         # 计算达标时间
260         if hasattr(kmf_bootstrap, '
survival_function_'):
261             sf = kmf_bootstrap.survival_function_
262         else:
263             continue
264
265         if len(sf) > 0:
266             times_bs = sf.index.values
267             probs_bs = sf.iloc[:, 0].values
268             if (probs_bs <= 0.2).any():
269                 idx = np.argmax(probs_bs <= 0.2)
270                 if idx > 0 and idx < len(times_bs)
:
271                     t1, p1 = times_bs[idx-1],
probs_bs[idx-1]
272                     t2, p2 = times_bs[idx],
probs_bs[idx]
273                     if p2 != p1: # 避免除零错误
274                         x_bs = t1 + (0.2 - p1) * (
t2 - t1) / (p2 - p1)
275                         bootstrap_estimates.append
(x_bs)
276         except:
277             continue
278
279         if len(bootstrap_estimates) > 10:
280             mean_estimate = np.mean(bootstrap_estimates)
281             std_estimate = np.std(bootstrap_estimates)
282             ci_lower = np.percentile(bootstrap_estimates,
2.5)
283             ci_upper = np.percentile(bootstrap_estimates,
97.5)
284
285         print(f"{cluster}: {x_value:.2f} ± {

```

```

std_estimate:.2f} 周")
286         print(f" 95%置信区间: [{ci_lower:.2f}, {
ci_upper:.2f}] 周")
287         print(f" 相对误差: ±{(std_estimate/x_value)
*100:.1f}%")
288     else:
289         print(f"{cluster}: Bootstrap样本不足, 无法估计
误差")
290     else:
291         print(f"{cluster}: {x_value}")
292
293 # 6. 特征分布稳定性检查
294 print("\n6. 特征分布稳定性检查:")
295 for feature in features:
296     print(f"\n特征 '{feature}' 在各聚类的分布:")
297     for cluster in range(best_k):
298         cluster_values = data_clean[data_clean['聚类标签']
== cluster][feature]
299         if len(cluster_values) > 0:
300             mean_val = cluster_values.mean()
301             std_val = cluster_values.std()
302             print(f" 聚类{cluster}: {mean_val:.2f} ± {
std_val:.2f}")
303         else:
304             print(f" 聚类{cluster}: 无数据")
305
306 # 7. 敏感性分析 (简化版本)
307 print("\n7. 敏感性分析:")
308 thresholds = [0.15, 0.20, 0.25]
309 for threshold in thresholds:
310     print(f"\n使用阈值 {threshold} (达标率 {1-threshold
:.0%}):")
311     for cluster in range(best_k):
312         cluster_data = data_clean[data_clean['聚类标签']
== cluster]

```



```

313         if len(cluster_data) > 0:
314             T = cluster_data['time']
315             E = (cluster_data['Y染色体浓度'] >= 0.04 * (
threshold/0.2)).astype(int)
316
317             kmf_temp = KaplanMeierFitter()
318             kmf_temp.fit(T, event_observed=E)
319
320             if hasattr(kmf_temp, 'survival_function_'):
321                 sf = kmf_temp.survival_function_
322                 if len(sf) > 0:
323                     times_temp = sf.index.values
324                     probs_temp = sf.iloc[:, 0].values
325                     if (probs_temp <= threshold).any():
326                         idx = np.argmax(probs_temp <=
threshold)
327                         if idx > 0 and idx < len(
times_temp):
328                             t1, p1 = times_temp[idx-1],
probs_temp[idx-1]
329                             t2, p2 = times_temp[idx],
probs_temp[idx]
330                             if p2 != p1:
331                                 x_temp = t1 + (threshold -
p1) * (t2 - t1) / (p2 - p1)
332                                 print(f" 聚类{cluster}: {
x_temp:.2f} 周")
333
334             # 8. 模型鲁棒性检查
335             print("\n8. 模型鲁棒性检查:")
336             print("各聚类样本数量统计:")
337             cluster_counts = data_clean['聚类标签'].value_counts().
sort_index()
338             for cluster, count in cluster_counts.items():
339                 print(f" 聚类{cluster}: {count} 个样本")

```

```

340         if count < 20:
341             print(f"    □ 样本量较少，结果可能不稳定")
342
343 # 执行误差分析
344 perform_error_analysis(data_clean, intersection_x, best_k)
345
346 # ----- 第七步：输出核心结果
347     -----
348 print("\n" + "="*60)
349 print("各聚类80%达标时间（生存曲线与80%达标线交点横坐标）")
350 print("="*60)
351 for cluster, x in intersection_x.items():
352     print(f"{cluster}: {x} 周")
353
354 print("\n□ 误差分析完成！")
355
356 # ----- 第八步：简化结果可视化模块
357     -----
358 def visualize_simplified_results(data_clean, intersection_x,
359     best_k, features):
360     """简化结果可视化函数 - 只生成2张核心图表"""
361     print("\n□□ 开始生成简化结果可视化图表...")
362
363     # 1. 聚类特征分布雷达图
364     # 计算每个聚类的特征均值
365     cluster_means = []
366     cluster_labels = []
367
368     for cluster in range(best_k):
369         cluster_data = data_clean[data_clean['聚类标签'] ==
370 cluster]
371         if len(cluster_data) > 0:
372             means = [cluster_data[feature].mean() for feature
373 in features]
374             cluster_means.append(means)

```

```

370         cluster_labels.append(f' 聚类{cluster}')
371
372     if len(cluster_means) == 0:
373         print("□ 无有效聚类数据，无法生成雷达图")
374         return
375
376     # 使用全局标准化方法
377     from sklearn.preprocessing import StandardScaler
378     scaler = StandardScaler()
379     cluster_means_scaled = scaler.fit_transform(cluster_means)
380
381     # 创建雷达图
382     angles = np.linspace(0, 2*np.pi, len(features), endpoint=
False).tolist()
383     angles += angles[:1]
384
385     fig, ax = plt.subplots(figsize=(10, 10), subplot_kw=dict(
polar=True))
386
387     # 使用颜色方案
388     colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '
#9467bd'] # 多种颜色
389
390     for i, (means_scaled, color) in enumerate(zip(
cluster_means_scaled, colors)):
391         if i >= len(colors):
392             color = colors[i % len(colors)]
393         values = means_scaled.tolist()
394         values += values[:1] # 闭合图形
395         ax.plot(angles, values, 'o-', linewidth=2, label=
cluster_labels[i], color=color)
396         ax.fill(angles, values, alpha=0.25, color=color)
397
398     # 设置角度标签
399     ax.set_thetagrids(np.degrees(angles[:-1]), features,

```

```

fontsize=14)
400     ax.set_ylim(-2, 2)
401
402     # 添加标题和网格
403     ax.set_title('各聚类特征均值示意图（标准化后）', size=16,
pad=25)
404     ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1),
fontsize=12)
405     ax.grid(True, alpha=0.3)
406
407     plt.tight_layout()
408     plt.savefig('聚类特征雷达图.png', dpi=300, bbox_inches='
tight')
409     plt.show()
410
411     # 2. 达标时间对比条形图
412     plt.figure(figsize=(12, 8))
413
414     # 提取数值型的达标时间
415     valid_times = []
416     valid_clusters = []
417
418     for cluster, time_val in intersection_x.items():
419         if isinstance(time_val, (int, float)):
420             valid_times.append(time_val)
421             valid_clusters.append(cluster)
422
423     if valid_times:
424         colors = plt.cm.viridis(np.linspace(0, 1, len(
valid_times)))
425         bars = plt.bar(valid_clusters, valid_times, color=
colors, alpha=0.7, edgecolor='black')
426
427         # 添加数值标签
428         for bar, time_val in zip(bars, valid_times):

```

```

429         plt.text(bar.get_x() + bar.get_width()/2, bar.
get_height() + 0.1,
430                 f'{time_val}周', ha='center', va='bottom',
fontsize=12, fontweight='bold')
431
432     plt.xlabel('聚类分组', fontsize=14, labelpad=10)
433     plt.ylabel('80%达标时间（周）', fontsize=14, labelpad
=10)
434     plt.title('各聚类80%达标时间对比', fontsize=16, pad
=20)
435     plt.grid(True, alpha=0.3, axis='y')
436
437     # 设置y轴范围，留出空间显示数值标签
438     y_max = max(valid_times) * 1.2
439     plt.ylim(0, y_max)
440
441     plt.xticks(rotation=45, fontsize=12)
442     plt.yticks(fontsize=12)
443     plt.tight_layout()
444     plt.savefig('达标时间对比.png', dpi=300, bbox_inches='
tight')
445     plt.show()
446
447     print("□ 简化可视化完成！已生成2张核心图表：")
448     print("1. 聚类特征雷达图.png")
449     print("2. 达标时间对比.png")
450
451     # 输出雷达图的原始数值
452     print("\n□□ 雷达图原始数值（标准化前）：")
453     for i, cluster_label in enumerate(cluster_labels):
454         print(f"\n{cluster_label}:")
455         for j, feature in enumerate(features):
456             print(f"    {feature}: {cluster_means[i][j]:.2f}
")
457     else:

```

```

458         print("□ 无有效达标时间数据，无法生成对比图")
459
460 # 执行简化结果可视化
461 visualize_simplified_results(data_clean, intersection_x,
462                               best_k, features)
463 # ----- 第九步：最终结果汇总
464 # -----
465 print("\n" + "="*60)
466 print("最终分析结果汇总")
467 print("="*60)
468
469 print(f"\n□□ 聚类分析结果:")
470 print(f"最佳聚类数: {best_k}")
471 print(f"轮廓系数: {max(silhouette_scores):.4f}")
472
473 print(f"\n□ 各聚类80%达标时间:")
474 for cluster, x in intersection_x.items():
475     print(f" {cluster}: {x} 周")
476
477 print(f"\n□□ 样本量分布:")
478 cluster_counts = data_clean['聚类标签'].value_counts().
479     sort_index()
480 for cluster, count in cluster_counts.items():
481     print(f" 聚类{cluster}: {count} 个样本")
482
483 print(f"\n□□ 临床建议:")
484 valid_times = [x for x in intersection_x.values() if
485                 isinstance(x, (int, float))]
486 if valid_times:
487     avg_time = np.mean(valid_times)
488     min_time = min(valid_times)
489     max_time = max(valid_times)
490     print(f" • 平均推荐检测时间: {avg_time:.1f} 周")
491     print(f" • 最早推荐时间: {min_time:.1f} 周")

```

```

489     print(f"    • 最晚推荐时间：{max_time:.1f} 周")
490     print(f"    • 建议检测时间范围：{min_time:.1f}-{max_time:.1f} 周")
491
492 print(f"\n□□ 解读说明:")
493 print("• 雷达图显示各聚类的特征分布模式（已标准化到0-1范围）")
494 print("• 条形图显示各聚类达到80%Y染色体浓度达标率所需时间")
495 print("• 可根据不同孕妇的特征模式，参考对应聚类的推荐检测时间"
496       )
497 print("\n□□ 分析完成！核心结果已保存为PNG图片文件。")

```

q4.py

```

1  import matplotlib.pyplot as plt
2  import pandas as pd
3  import numpy as np
4  from sklearn.model_selection import train_test_split
5  from sklearn.ensemble import RandomForestClassifier
6  from sklearn.metrics import classification_report,
   confusion_matrix, roc_auc_score, accuracy_score
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.impute import SimpleImputer
9  import seaborn as sns
10 import warnings
11 warnings.filterwarnings('ignore')
12
13 # ===== 中文乱码解决方案 =====
14 def setup_chinese_font():
15     """设置中文字体，解决乱码问题"""
16     import matplotlib as mpl
17     from matplotlib import font_manager
18     import platform
19     import os
20
21     # 重置matplotlib默认设置

```

```

22     mpl.rcParams.update(mpl.rcParamsDefault)
23
24     # 设置样式
25     sns.set_style("whitegrid")
26     plt.rcParams['figure.figsize'] = [10, 6]
27     plt.rcParams['font.size'] = 12
28     plt.rcParams['axes.unicode_minus'] = False # 解决负号显示
问题
29
30     # 根据不同操作系统设置字体
31     system = platform.system()
32
33     # Windows系统
34     if system == "Windows":
35         font_paths = [
36             "C:/Windows/Fonts/simhei.ttf",      # 黑体
37             "C:/Windows/Fonts/msyh.ttc",        # 微软雅黑
38             "C:/Windows/Fonts/simsun.ttc",      # 宋体
39             "C:/Windows/Fonts/simkai.ttf",      # 楷体
40         ]
41     # 尝试加载字体文件
42     for font_path in font_paths:
43         if os.path.exists(font_path):
44             try:
45                 # 添加到字体管理器
46                 font_prop = font_manager.FontProperties(fname=
font_path)
47                 font_manager.fontManager.addfont(font_path)
48
49                 # 设置全局字体
50                 plt.rcParams['font.family'] = [font_prop.
get_name(), 'DejaVu Sans', 'Arial']
51                 print(f"□ 成功设置字体: {font_prop.get_name()}
")
52                 return True

```



```

53         except Exception as e:
54             continue
55
56     # 如果字体文件加载失败，尝试使用字体名称
57     font_names = [
58         'SimHei', 'Microsoft YaHei', 'SimSun', 'NSimSun',
59         'FangSong', 'KaiTi', 'STSong', 'STKaiti',
60         'PingFang SC', 'Hiragino Sans GB', 'WenQuanYi Micro
61     Hei'
62     ]
63
64     for font_name in font_names:
65         try:
66             plt.rcParams['font.family'] = [font_name, 'DejaVu
67     Sans', 'sans-serif']
68             # 测试字体是否可用
69             fig, ax = plt.subplots()
70             ax.text(0.5, 0.5, '测试中文', fontsize=12, ha='
71     center')
72             plt.close(fig)
73             print(f"□ 成功设置字体: {font_name}")
74             return True
75         except:
76             continue
77
78     print("□ 无法设置中文字体，将使用英文显示")
79     plt.rcParams['font.family'] = ['DejaVu Sans', 'Arial', '
80     sans-serif']
81     return False
82
83 # 调用字体设置函数
84 setup_chinese_font()
85
86 # ===== 数据读取与预处理 =====
87 print("开始数据读取与预处理...")

```

```

84 file_path = r"C:\Users\27645\Desktop\女.xlsx"
85 df = pd.read_excel(file_path)
86
87 print(f"数据形状: {df.shape}")
88
89 # 创建目标变量
90 df['abnormal'] = df['染色体的非整倍体'].notna().astype(int)
91 print(f"\n异常样本数量: {df['abnormal'].sum()}")
92 print(f"总样本数量: {len(df)}")
93
94 # 选择特征
95 features = [
96     '年龄', '体重', '孕妇BMI', '身高',
97     '原始读段数', '在参考基因组上比对的比例', '重复读段的比例'
98     ,
99     '唯一比对的读段数', 'GC含量', '被过滤掉读段数的比例',
100     '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
101     'X染色体的Z值', 'X染色体浓度',
102     '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的
    GC含量',
103     '怀孕次数', '生产次数'
104 ]
105 # 准备特征矩阵
106 X_original = df[[f for f in features if f in df.columns]].copy()
107 y = df['abnormal']
108
109 # 转换数据类型
110 print("\n转换数据类型为数值型...")
111 X_model = X_original.copy()
112 for column in X_model.columns:
113     X_original[column] = pd.to_numeric(X_original[column],
114     errors='coerce')
115     X_model[column] = pd.to_numeric(X_model[column], errors='

```

```

    coerce')
115
116 print(f"特征矩阵形状: {X_model.shape}")
117 print(f"目标变量分布:\n{y.value_counts()}")
118
119 # 处理缺失值
120 print(f"\n缺失值统计:")
121 print(X_model.isnull().sum())
122
123 imputer = SimpleImputer(strategy='mean')
124 X_model_imputed = imputer.fit_transform(X_model)
125 X_model = pd.DataFrame(X_model_imputed, columns=X_model.
    columns, index=X_model.index)
126
127 X_original_imputed = imputer.transform(X_original)
128 X_original = pd.DataFrame(X_original_imputed, columns=
    X_original.columns, index=X_original.index)
129
130 # 数据标准化
131 scaler = StandardScaler()
132 X_model_scaled = scaler.fit_transform(X_model)
133 X_model = pd.DataFrame(X_model_scaled, columns=X_model.columns
    , index=X_model.index)
134
135 # 划分训练集和测试集
136 X_train, X_test, y_train, y_test, idx_train, idx_test =
    train_test_split(
137     X_model, y, X_model.index, test_size=0.3, random_state=42,
    stratify=y
138 )
139
140 X_original_test = X_original.loc[idx_test].copy()
141
142 print(f"\n训练集形状: {X_train.shape}")
143 print(f"测试集形状: {X_test.shape}")

```

```

144
145 # ===== 模型训练与预测 =====
146 print("\n开始训练模型...")
147 rf_model = RandomForestClassifier(
148     n_estimators=100,
149     max_depth=8,
150     min_samples_split=5,
151     min_samples_leaf=2,
152     class_weight='balanced',
153     random_state=42,
154     n_jobs=-1
155 )
156
157 rf_model.fit(X_train, y_train)
158 print("模型训练完成!")
159
160 # 预测
161 y_pred = rf_model.predict(X_test)
162 y_pred_proba = rf_model.predict_proba(X_test)[:, 1]
163
164 # 模型评估
165 print("=" * 50)
166 print("模型性能评估")
167 print("=" * 50)
168 print(f"准确率: {accuracy_score(y_test, y_pred):.4f}")
169
170 if len(np.unique(y_test)) > 1:
171     print(f"AUC分数: {roc_auc_score(y_test, y_pred_proba):.4f}")
172
173 # 特征重要性分析
174 feature_importance = pd.DataFrame({
175     'feature': X_model.columns,
176     'importance': rf_model.feature_importances_
177 }).sort_values('importance', ascending=False)

```

```

178
179 print("\nTop 5重要特征:")
180 print(feature_importance.head(5))
181
182 # ===== 简化可视化 =====
183 def reverse_standardize(scaled_data, scaler, feature_names):
184     """去标准化函数"""
185     original_data = scaler.inverse_transform(scaled_data)
186     return pd.DataFrame(original_data, columns=feature_names,
187                          index=scaled_data.index)
187
188 # 定位异常样本
189 abnormal_pred_idx = X_test[y_pred == 1].index
190 print(f"\n测试集中预测为异常的样本数量: {len(abnormal_pred_idx)}")
191
192 if len(abnormal_pred_idx) == 0:
193     print("□ 测试集中无预测为异常的样本，无法生成图表")
194 else:
195     # 提取异常样本数据
196     abnormal_scaled_features = X_test.loc[abnormal_pred_idx].
197     copy()
198     abnormal_original_features = reverse_standardize(
199         abnormal_scaled_features, scaler, X_model.columns
200     )
201     abnormal_original_features['真实标签'] = y_test.loc[
202         abnormal_pred_idx].values
203     abnormal_original_features['异常概率'] = y_pred_proba[
204         y_pred == 1]
205
206     # 获取Top4重要特征
207     top4_features = feature_importance['feature'].head(4).
208     tolist()
209     print(f"\n用于可视化的Top4特征: {top4_features}")
210

```

```

207 # 创建三个子图，每个包含两个图表
208 print("\n生成组合图表...")
209
210 # 图1：两个特征分布对比图
211 fig1, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
212
213 # 第一个特征
214 feature1 = top4_features[0]
215 normal_values1 = X_original.loc[X_train.index][feature1]
216 abnormal_values1 = abnormal_original_features[feature1]
217
218 ax1.hist(normal_values1, bins=30, alpha=0.6, color='green'
219 ,
220         label=f'正常样本 (n={len(normal_values1)})',
221         density=True)
222 ax1.hist(abnormal_values1, bins=15, alpha=0.6, color='red'
223 ,
224         label=f'异常样本 (n={len(abnormal_values1)})',
225         density=True)
226 ax1.set_xlabel(feature1, fontsize=12)
227 ax1.set_ylabel('密度', fontsize=12)
228 ax1.legend(fontsize=10)
229 ax1.grid(True, alpha=0.3)
230
231 # 第二个特征
232 feature2 = top4_features[1]
233 normal_values2 = X_original.loc[X_train.index][feature2]
234 abnormal_values2 = abnormal_original_features[feature2]
235
236 ax2.hist(normal_values2, bins=30, alpha=0.6, color='green'
237 ,
238         label=f'正常样本 (n={len(normal_values2)})',
239         density=True)
240 ax2.hist(abnormal_values2, bins=15, alpha=0.6, color='red'
241 ,

```

```

235         label=f'异常样本 (n={len(abnormal_values2)})',
density=True)
236     ax2.set_xlabel(feature2, fontsize=12)
237     ax2.set_ylabel('密度', fontsize=12)
238     ax2.legend(fontsize=10)
239     ax2.grid(True, alpha=0.3)
240
241     plt.suptitle('特征分布对比', fontsize=16)
242     plt.tight_layout(rect=[0, 0, 1, 0.96]) # 为总标题留出空间
243     plt.savefig('特征分布对比1.png', dpi=300, bbox_inches='
tight')
244     plt.show()
245
246     # 图2: 另外两个特征分布对比图
247     fig2, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
248
249     # 第三个特征
250     feature3 = top4_features[2]
251     normal_values3 = X_original.loc[X_train.index][feature3]
252     abnormal_values3 = abnormal_original_features[feature3]
253
254     ax1.hist(normal_values3, bins=30, alpha=0.6, color='green'
,
255         label=f'正常样本 (n={len(normal_values3)})',
density=True)
256     ax1.hist(abnormal_values3, bins=15, alpha=0.6, color='red'
,
257         label=f'异常样本 (n={len(abnormal_values3)})',
density=True)
258     ax1.set_xlabel(feature3, fontsize=12)
259     ax1.set_ylabel('密度', fontsize=12)
260     ax1.legend(fontsize=10)
261     ax1.grid(True, alpha=0.3)
262
263     # 第四个特征

```

```

264     feature4 = top4_features[3]
265     normal_values4 = X_original.loc[X_train.index][feature4]
266     abnormal_values4 = abnormal_original_features[feature4]
267
268     ax2.hist(normal_values4, bins=30, alpha=0.6, color='green'
269 ,
270             label=f'正常样本 (n={len(normal_values4)})',
271             density=True)
272     ax2.hist(abnormal_values4, bins=15, alpha=0.6, color='red'
273 ,
274             label=f'异常样本 (n={len(abnormal_values4)})',
275             density=True)
276
277     ax2.set_xlabel(feature4, fontsize=12)
278     ax2.set_ylabel('密度', fontsize=12)
279     ax2.legend(fontsize=10)
280     ax2.grid(True, alpha=0.3)
281
282     plt.suptitle('特征分布对比', fontsize=16)
283     plt.tight_layout(rect=[0, 0, 1, 0.96])
284     plt.savefig('特征分布对比2.png', dpi=300, bbox_inches='
285 tight')
286     plt.show()
287
288     # 图3: 两个异常概率分布图
289     fig3, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
290
291     # 所有测试样本异常概率分布
292     ax1.hist(y_pred_proba, bins=30, alpha=0.7, color='blue',
293             edgecolor='black')
294     ax1.axvline(x=0.5, color='red', linestyle='--', label='阈
295 值 (0.5)', linewidth=2)
296     ax1.set_xlabel('异常预测概率', fontsize=12)
297     ax1.set_ylabel('样本数量', fontsize=12)
298     ax1.legend(fontsize=10)
299     ax1.grid(True, alpha=0.3)

```



```

292     ax1.set_title('所有测试样本异常概率分布', fontsize=14)
293
294     # 异常样本概率分布
295     abnormal_probs = abnormal_original_features['异常概率']
296     ax2.hist(abnormal_probs, bins=15, alpha=0.7, color='red',
edgecolor='black')
297     ax2.set_xlabel('异常预测概率', fontsize=12)
298     ax2.set_ylabel('异常样本数量', fontsize=12)
299     ax2.grid(True, alpha=0.3)
300     ax2.set_title('异常样本概率分布', fontsize=14)
301
302     plt.suptitle('异常概率分布', fontsize=16)
303     plt.tight_layout(rect=[0, 0, 1, 0.96])
304     plt.savefig('异常概率分布.png', dpi=300, bbox_inches='
tight')
305     plt.show()
306
307     print("\n□ 可视化完成! 已生成3张组合图表: ")
308     print("1. 特征分布对比1.png")
309     print("2. 特征分布对比2.png")
310     print("3. 异常概率分布.png")
311
312     print("\n□□ 分析完成! ")

```